



Институт информационных технологий
Кафедра цифровой трансформации

Проектирование баз данных

*Фундамент, на котором строятся приложения
и который состоит из данных*

Лекция №1. Базы данных и управление ими

Дзгоев Алан Эдуардович

Кандидат технических наук,
доцент кафедры цифровой
трансформации

Институт информационных технологий
РТУ МИРЭА

Москва,
17.02.2025



СВЕДЕНИЯ О ДИСЦИПЛИНЕ

Лекции ведет:

- Кандидат технических наук,
доцент каф. Цифровой трансформации Дзгоев Алан Эдуардович
(Dzgoyev@mirea.ru / *При обращении в теме письма указывайте
номер группы*)

Объём **аудиторной** работы по дисциплине в 4 семестре:

- Лекции – 16 часов;
- Практические занятия – 32 часа;
- Аттестация – экзамен.

Допуск к экзамену:

- посещение лекций и практических работ;
- выполнение в установленный срок всех практических работ.

подробности в памятке по дисциплине (следующие слайды)

Показатель	Л4_ % кликов по кнопке "контроль присутствия"	Л5_ % кликов по кнопке "контроль присутствия"	Л6_04-05_ % присутствия	Л7_18-05_ % присутствия	Л8_01-06_ % присутствия	%СРЗНАЧ участия в лекциях	Задание:Практика 1_Законодательная и нормативно-техническая база стандартизации в РФ	Задание:Практика 2_Нормативные документы по стандартизации ИТ	Задание:Практика 3_ч1_Классификаторы в области ИТ	Задание:Практика 3_ч2_Классификаторы в области ИТ	Задание:Практика 3_ч3_Классификаторы в области ИТ	Задание:Практика 4_ч1_Создание технического задания в соответствии с ГОСТ 34 602–2020
0	0	50				6,3	-	-	-	-	-	-
0	0	50		100	100	31,3	-	1	1	1	1	1
0	0	0				0	-	-	-	-	-	-
100	0	50	100		100	56,3	1	1	1	1	1	1
100	100	100	100	100	100	87,5	1	1	1	1	1	1
0	0	0				0	-	-	-	-	-	-
0	50	0				6,3	1	-	-	-	-	-
100	0	100		100		56,3	1	1	1	1	1	1
0	0	0				12,5	1	1	-	-	-	-
0	0	0				0	-	-	-	-	-	-
0	0	0				0	-	-	-	-	-	-
100	0	0				18,8	-	-	-	-	-	-
0	100	100	100	100	100	68,8	1	1	1	1	1	1
0	0	50				18,8	-	-	-	-	-	-
0	0	0	100	100	100	50	1	1	1	1	1	1
0	0	0				0	-	-	-	-	-	-
0	100	100	100	100	100	100	1	1	1	1	1	1
0	0	0				0	-	-	-	-	-	-
100	0	0				18,8	1	1	1	1	-	-
50	0	50				18,8	-	-	-	-	-	-
0	0	0				0	-	-	-	-	-	-
0	0	0				0	-	-	-	-	-	-
0	0	0			100	12,5	1	1	1	1	1	1
100	100	0	0			25	1	1	-	-	-	-
0	0	0				0	-	-	-	-	-	-
100	100	100	100	100	100	100	1	1	1	1	1	1

Рекомендуемая литература

1. Новиков Б.А. Основы технологий баз данных: учебное пособие / Б.А. Новиков, Е.А. Горшкова, Н.Г. Графеева; под.ред. Е.В. Рогова. – 2-е изд. – М.: ДМК Пресс, 2020. – 582 с. Режим доступа: <https://postgrespro.ru/education/books/dbtech>
2. Моргунов Е.П. PostgreSQL. Основы языка SQL: учеб. Пособие / Е.П. Моргунов; под. ред. Е.В. Рогова, П.В. Лузанова. – СПб.: БХВ-Петербург, 2018. – 336 с.: ил. Режим доступа: <https://postgrespro.ru/education/books/sqlprimer>
3. Комаров В.И. Путеводитель по базам данных. – М.: ДМК-Пресс, 2024. – 520 с. Режим доступа: <https://edu.postgrespro.ru/dbguide.pdf>
4. Смирнов М.В. Проектирование баз данных [Электронный ресурс]: Конспект лекций / Смирнов М.В. - М., МИРЭА – Российский технологический университет, 2020 – 1 электрон. Опт. диск (CD-ROM). Режим доступа: <https://e.lanbook.com/book/163892>
5. Чистякова М.А. Проектирование и эксплуатация баз данных [Электронный ресурс]: Учебно-методическое пособие / Чистякова М.А., Иванова И.А., Котилевец И.Д. – М.: МИРЭА – Российский технологический университет, 2021. – 1 электрон. Опт. диск (CD-ROM). Режим доступа: <https://e.lanbook.com/book/176572>
6. Братусь Н.В. Базы данных. Часть 1 [Электронный ресурс]: Практикум / Братусь Н.В., Маличенко С.В., Матчин В.Т. — М.: МИРЭА – Российский технологический университет, 2024. — 1 электрон. опт. диск (CD-ROM) – Режим доступа: <https://ibc.mirea.ru/books/SHARE/5859/>
7. Смирнов М.В. Проектирование хранилищ данных [Электронный ресурс]: Учебно-методическое пособие / Смирнов М.В. — М.: МИРЭА – Российский технологический университет, 2024. — 1 электрон. опт. диск (CD-ROM)– Режим доступа: <https://ibc.mirea.ru/books/SHARE/55283/>

Неделя	Лекции	Практические работы	Контр.меропр.
1	Лекция 1. Базы данных и управление ими 2 часа	Практика 1. Создание модели предметной области в BPMN (<i>основы BPMN</i>) 2 часа	
2		Практика 1. 2 часа (дедлайн сдачи практики)	
3	Лекция 2. Реляционная структура данных 2 часа	Практика 2. Реляционная модель данных 2 часа	
4		Практика 2. 2 часа (дедлайн сдачи практики)	
5	Лекция 3. Жизненный цикл, модель данных «сущность-связь», нотации 2 часа	Практика 3. Создание DFD модели данных (Ramus) 2 часа	
6		Практика 3. 2 часа (дедлайн сдачи практики)	Тест №1 в СДО (на 20 минут на практике)
7	Лекция 4. Методология проектирования баз данных 2 часа	Практика 4. Концептуальная модель данных (ChartDB) 2 часа	
8		Практика 4. 2 часа (дедлайн сдачи практики)	
9	Лекция 5. Нормализация 2 часа	Практика 5. Логическая модель данных (ChartDB) 2 часа	
10		Практика 5. 2 часа	
11	Лекция 6. Физическая модель данных 2 часа	Практика 5. 2 часа (дедлайн сдачи практики)	
12		Практика 6. Физическая модель данных (ChartDB) 2 часа	
13	Лекция 7. Основы языка структурированных запросов (SQL). 2 часа	Практика 6. 2 часа (дедлайн сдачи практики)	

План – график лекций и практических занятий

14		Практика 6. 2 часа (написание контрольной)	Контр.работа (вся пара)
15	Лекция 8. Архитектура СУБД и корпоративная архитектура 2 часа (тест №3 в СДО)	Практика 7. Основы языка структурированных запросов (SQL) (Dbeaver + PostgrePRO по модели DBaaS) 2 часа	Тест №2 в СДО (на 20 минут, на практике)
16		Практика 7. 2 часа. (дедлайн сдачи практики)	
Итого	16 часов	32 часа	

ПАМЯТКА

о правилах обучения дисциплины

«Проектирование баз данных»

Наименование	Формат	Баллы (макс.)
Защита практических работ	Очно	42 (6 баллов за одну практику)
Контрольный тест №1 (Лекции №1-3, 6 неделя на практике)	СДО	8
Контрольный тест №2 (Лекции №4-6, 15 неделя на практике)	СДО	8
Контрольный тест №3 (Лекции №1-8, 15 неделя на лекции)	СДО	8
Контрольная работа (14 неделя)	Очно	10
Посещение (Лекции и практики)	Очно	24 (1 балл за посещение)
Максимальная сумма баллов		100

ДИСЦИПЛИНА	Проектирование баз данных (укажите полное наименование дисциплины без сокращений)
ИНСТИТУТ	информационных технологий (укажите название учебного института)
КАФЕДРА	Цифровой трансформации (укажите полное наименование кафедры)
Уровень обучения	Бакалавриат
Аудиторные часы	16/0/32

Лекции 16 часов.

Обязательное посещение всех лекций.

В тестах в рамках мероприятий текущего контроля вопросы из лекций.

Практические работы (7 работ)**Обязательное посещение всех практических занятий.****Допуск к экзамену – очная защита всех практических работ в течение семестра. Защита отчёта до дедлайна****Защита практической работы после дедлайна – 0 баллов, но работа засчитывается.**

Если есть справка по перенесённой болезни, заверенная в учебном отделе, то эту справку необходимо принести и показать преподавателю по практике. В таком случае назначается пересдача пропущенной практической работы в день консультаций (важно: студент защищает именно ту практическую работу, которую он пропустил по болезни, согласно датам в справке). Если студент пропустил практические работы без уважительной причины и, соответственно, работу не защитил, то на экзамен он не допускается.

Если студент загрузил отчет до дедлайна, но не защитил, то такой отчет не засчитывается студенту, т.е. не сдал.

Дополнительного времени на защиту практических работ не выделяется.

Выполненные работы студент загружает в СДО в формате pdf. Фон области построения модели – белый (в отчёте).

Наименование	Формат	Баллы (макс.)
Защита практических работ	Очно	42 (6 баллов за одну практику)
Контрольный тест №1 (Лекции №1-3, 6 неделя на практике)	СДО	8
Контрольный тест №2 (Лекции №4-6, 15 неделя на практике)	СДО	8
Контрольный тест №3 (Лекции №1-8, 15 неделя на лекции)	СДО	8
Контрольная работа (14 неделя)	Очно	10
Посещение (Лекции и практики)	Очно	24 (1 балл за посещение)
Максимальная сумма баллов		100

Если студент не загрузил отчет до дедлайна, то загрузка после дедлайна станет недоступна.

Обязательно, при составлении и оформлении текущего отчёта по практической работе, студент вставляет рисунок (схему, модель БД) из предыдущего отчета.

В отчёте по практике №2 (реляционная структура данных) необходимо набирать формулы, используя встроенный в Word формульный редактор, либо Equation 3.0, MathType.

Контрольные тесты

Тесты студенты выполняют в СДО на паре.

Баллы минусуются, если по базовым вопросам студент отвечает не правильно.

Проходного балла нет, сколько студент набрал столько и остается.

Контрольный тест №1. (Лекции 1, 2, 3) проводится на 6 неделе в начале пары (20 минут) на практике.

Контрольный тест №2. (Лекции 4, 5, 6) проводится на 15 неделе в начале пары (20 минут) на практике.

Контрольный тест №3 (Лекции №7-8) проводится на 15 неделе в начале пары на лекции.

Контрольная работа

Выполняется на 14 неделе в аудитории.

Студент получает задание на паре.

Время выполнения контрольной работы – 1 час.

Спроектированную модель данных студент импортирует в формате JSON и загружает в СДО до окончания пары. Если студент, не успел загрузить модель до окончания пары, доступ для загрузки JSON-документа закрывается.

Экзамен

Допуск к экзамену – защита всех практических работ в течение семестра очно. В случае, если студент не сдал какие-либо практики, у него есть возможность сдать их во время экзамена, если успеет. В противном случае, отправляется на пересдачу (необходимость сдачи работ сохраняется).

Если студент набрал меньше 50 % от общей суммы баллов, то экзамен будет проходить по билету (3 теоретических вопроса + 1 задача).

Если студент набрал больше 50% от суммы максимальной суммы баллов, то сдаёт тестирование на экзамене. + добавляются баллы за активность.

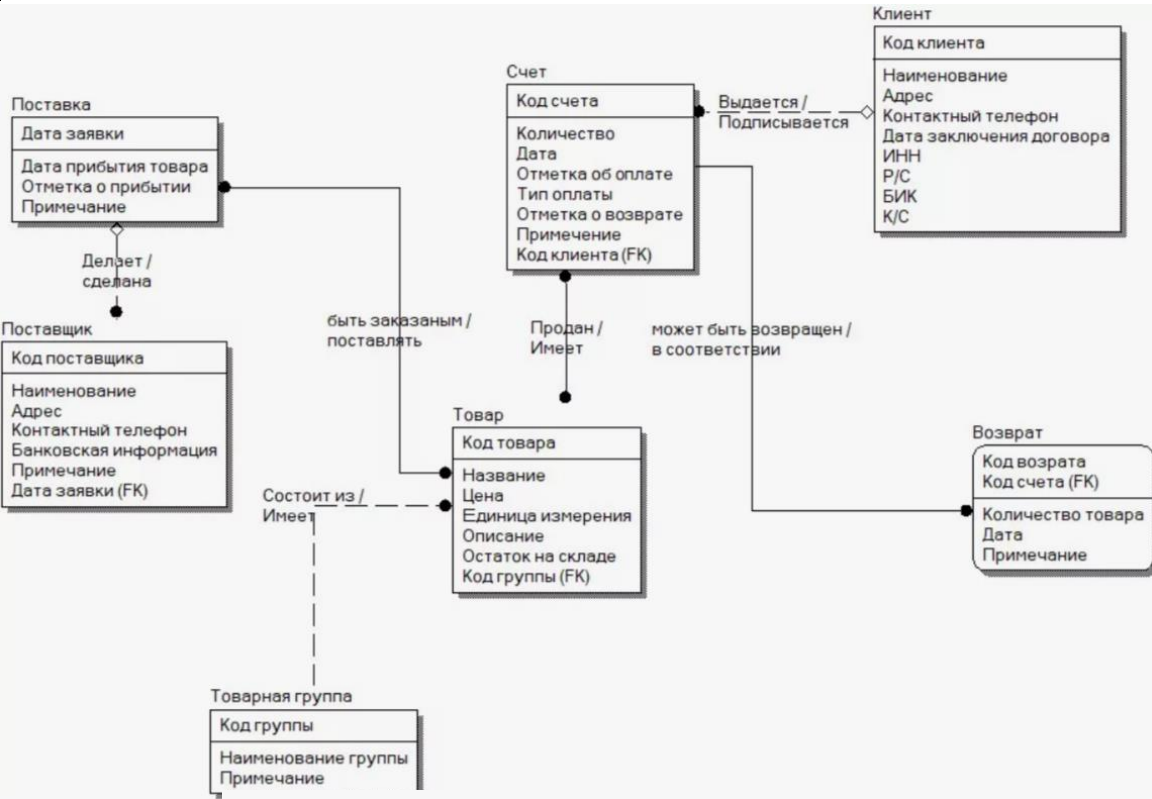
За тест студент может набрать 20 баллов, за которые можно получить:

- От 10 до 14 баллов — оценка «3»;
- От 15 до 19 баллов — оценка «4»;
- 20 баллов — оценка «5».

К баллам за тест прибавляются доп. баллы, которые студент получает в течение семестра по данным критериям:

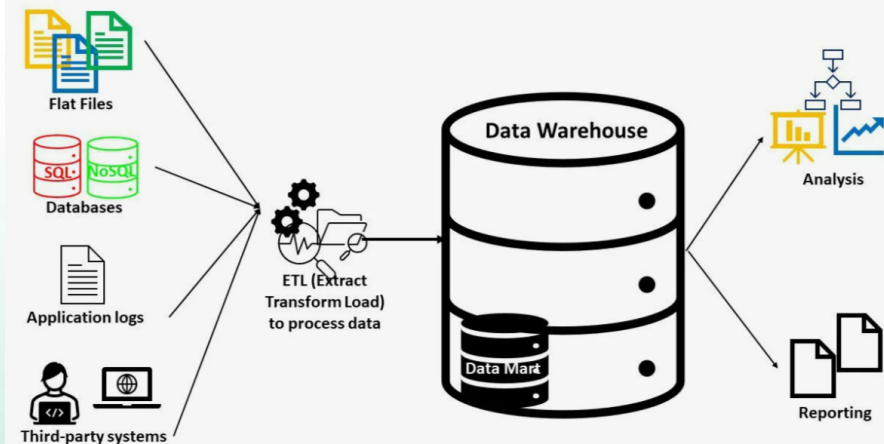
- Набрано 55-64 балла за семестр — 1 доп.;
- Набрано 65-74 балла за семестр — 2 доп.;
- Набрано 75-84 балла за семестр — 3 доп.;
- Набрано 85-94 балла за семестр — 4 доп.;
- Набрано 95-100 балла за семестр — 5 доп.

В случае, если студент сдал тест на «2», то отправляется на пересдачу без учета доп. баллов.

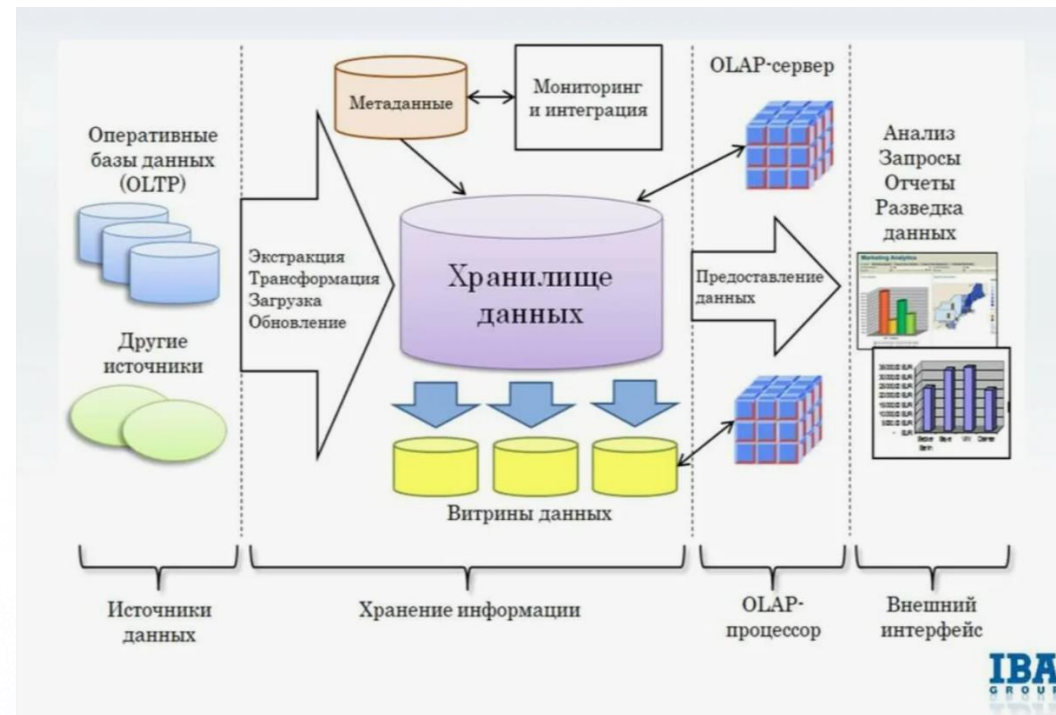
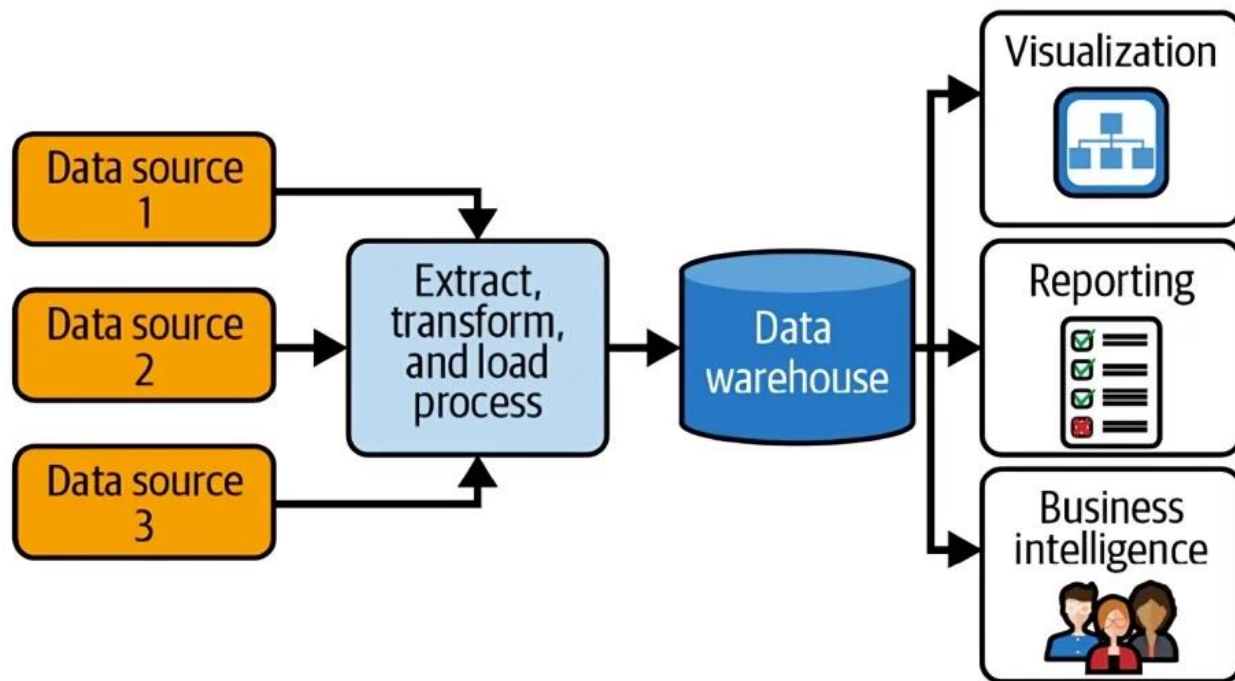


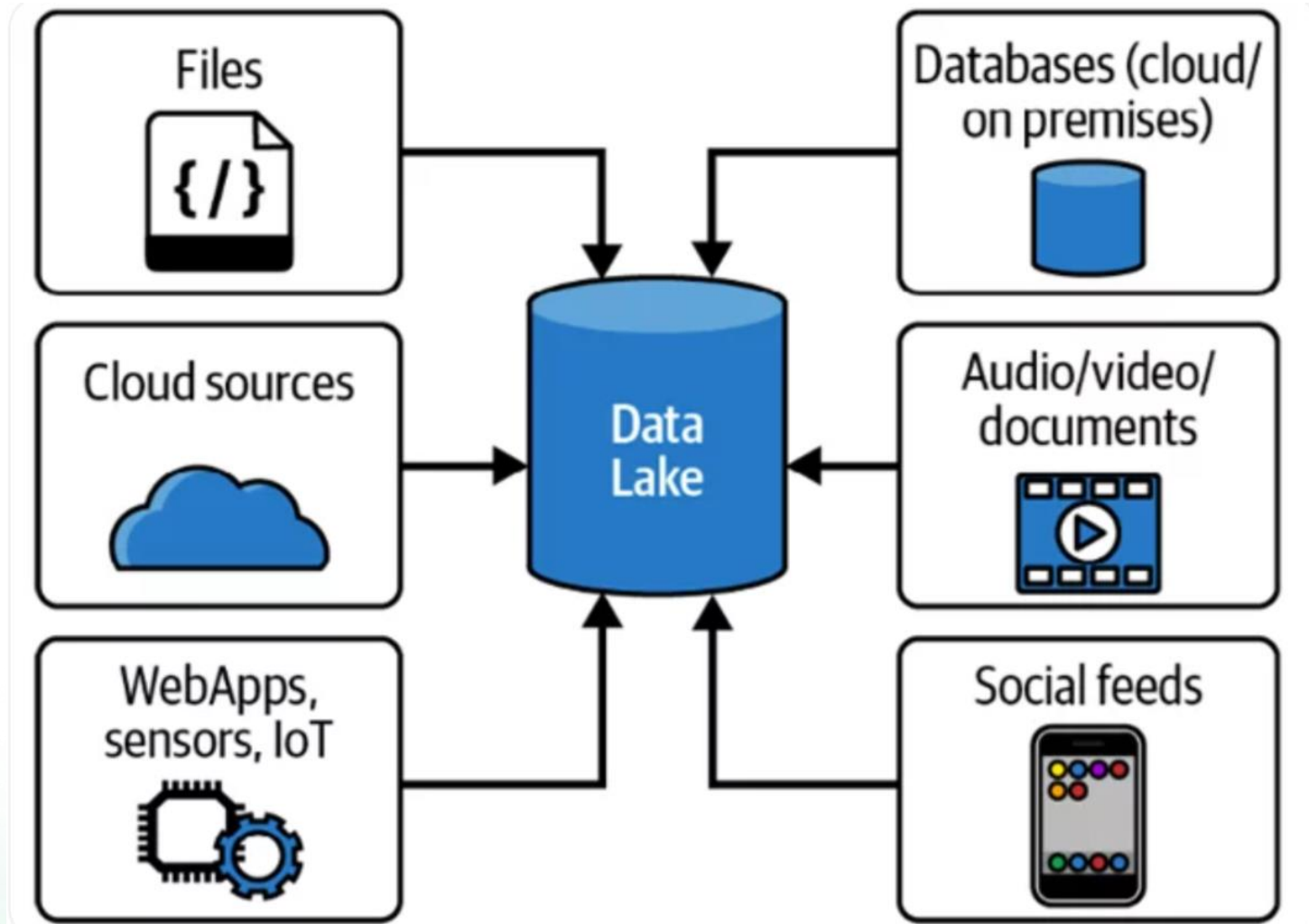
2 курс. Проектирование баз данных

3 курс. Разработка баз данных



Магистратура Проектирование и разработка баз и хранилищ данных





„В частном обслуживании наиболее перспективны с точки зрения ресурсоотдачи — это три составляющие. Первое — технология дистанционного обслуживания. Второе — работа с базами данных. И, наконец, третье — лучшие кадры.

Всё вместе это должно сформировать интеллектуальный капитал, то есть главный актив будущего неценового конкурентного преимущества“.

Дмитрий Васильевич Брейтенбихер

Источник: <https://ru.citaty.net/temy/bazy-dannykh/>

Введение в
проектирование
баз данных

Тема 1_ Базы данных и управление ими

1.1_Данные. Структуры данных.

1.2_Базы данных

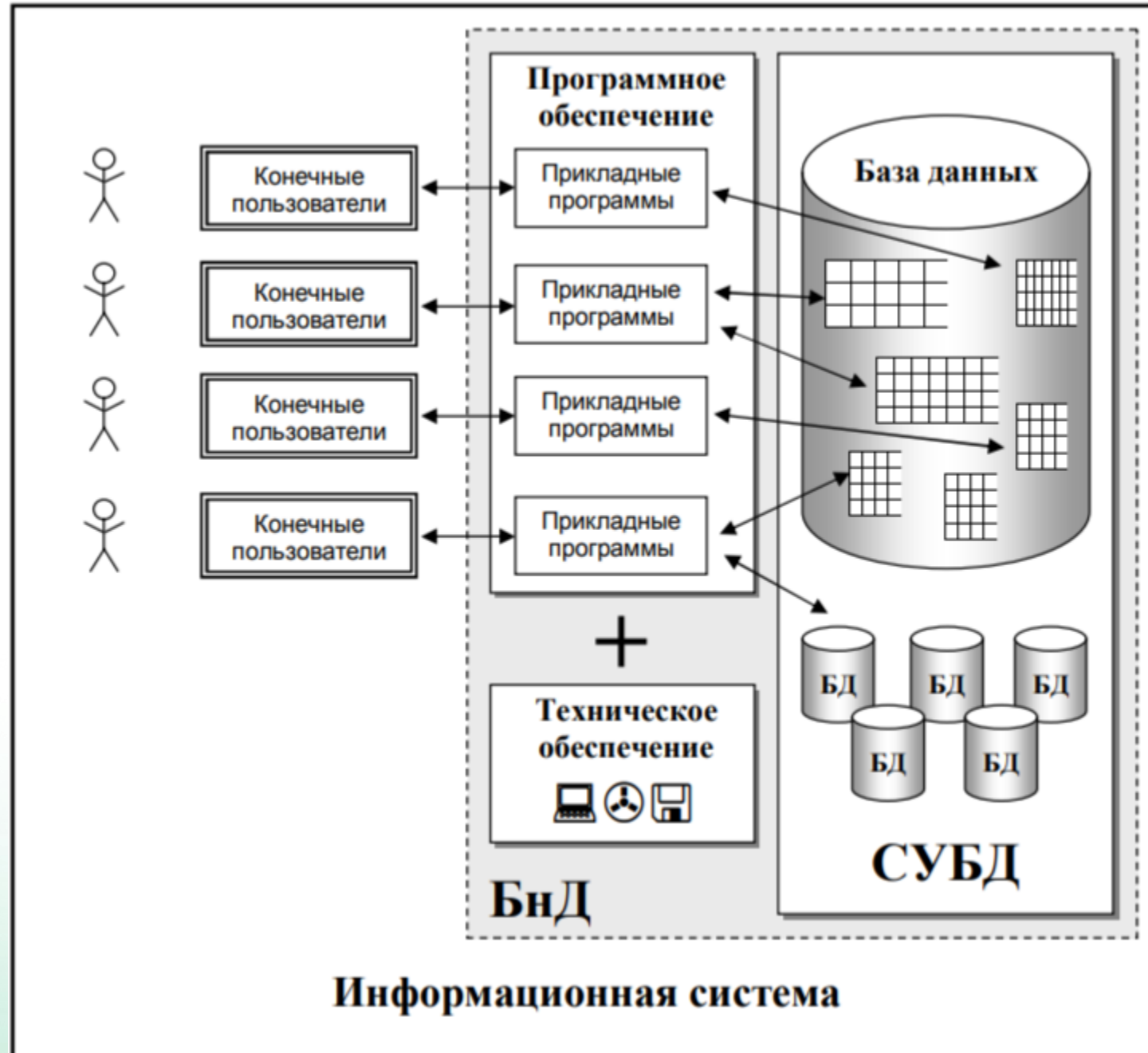
1.3_СУБД

1.4_Типы данных

История баз данных

- Середина 1960-х гг. Иерархические СУБД (IMS компании IBM)
- Середина 1960-х гг. Сетевые СУБД (IDMS/R компании Computer Associates)
- 1970 г. Доктор Е. Ф. Codd предложил реляционную модель.
- 1970-е гг. Предложены первые реляционные СУБД (Ingres в Berkeley и System R в компании IBM, язык SQL).
- 1976 г. П. Чен предложил модель «сущность–связь» (entity–relation) – ER-модель.
- 1979 г. Появление коммерческих реляционных СУБД Oracle, Ingres, DB2
- 1987 г. Стандарт ISO языка SQL (последующие выпуски стандарта: 1989, 1992 (SQL2), 1999 (SQL:1999), 2003 (SQL:2003), 2008 (SQL:2008), 2011 (SQL:2011))
- 1990-е гг. Появление объектно-ориентированных СУБД и объектно-реляционных СУБД
- 1990-е гг. Появление хранилищ данных (data warehousing)
- Середина 1990-х гг. Интеграция web с базами данных
- Середина 1990-х гг. – по настоящее время. Развитие open source СУБД (PostgreSQL, MySQL и др.)

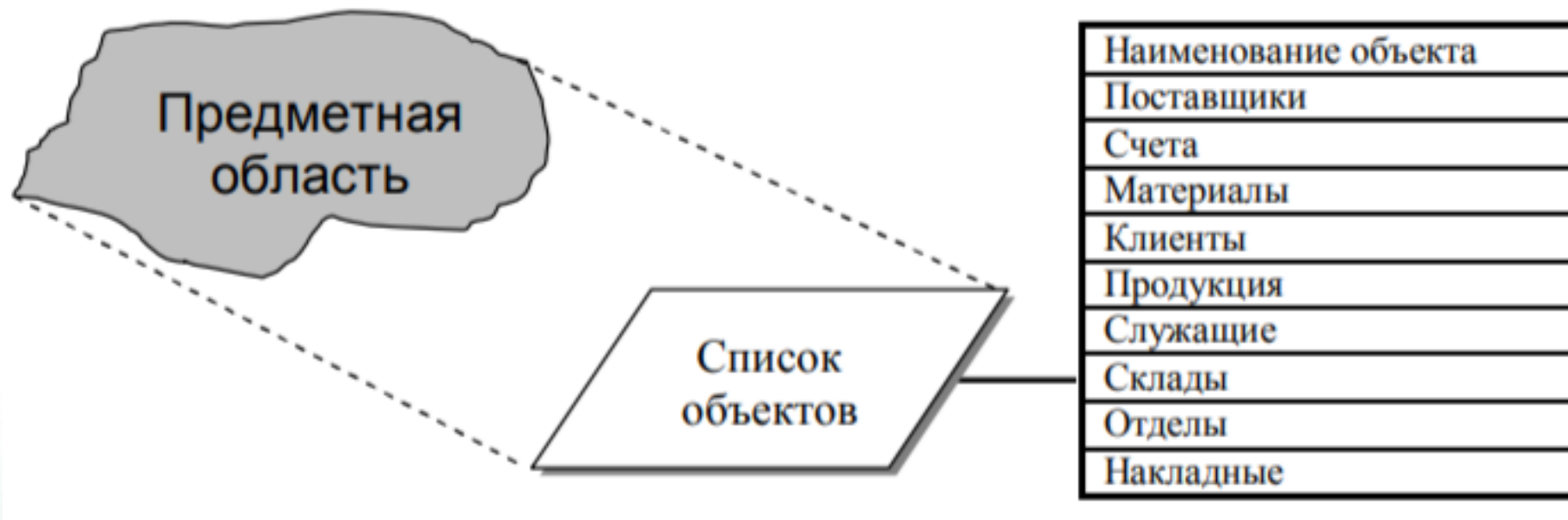
Тема 1_ Базы данных и управление ими



Тема 1_ Базы данных и управление ими

В связи с тем, что база данных является проекцией-отображением предметной области, для ее создания необходимо выделить из предметной области некоторые объекты. Свойства и состояния этих объектов будут отражаться в базе данных.

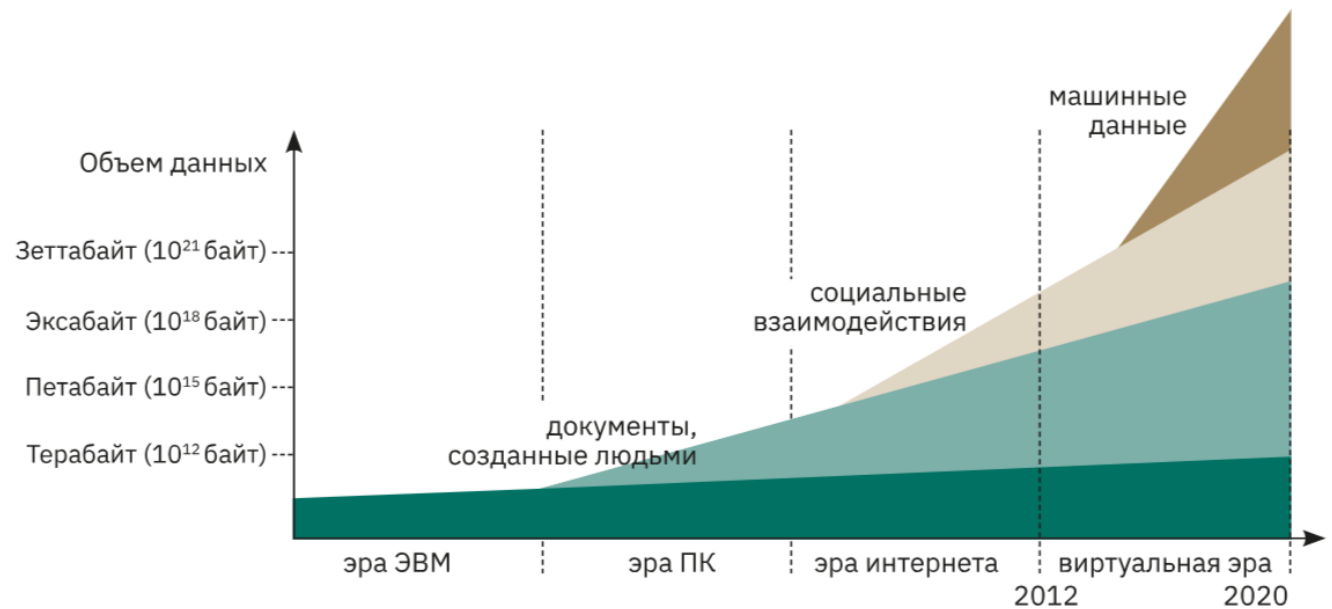
Выбор объектов предметной области



Данные

В английский язык слово information пришло в конце XIV века из французского (первоисточник — латинское informatio 'разъяснение, истолкование, сообщение'), а с середины XV века за английским information закрепилось значение 'переданные сведения, относящиеся к определенной теме'. Слово data в значении, близком к современному, стало использоваться в английском языке значительно позже. Оно происходит от латинского datum — 'данная вещь' (от глагола dare — 'давать').

В XVII веке благодаря быстрому распространению книгопечатания появилось множество научных книг, стремительно вырос объем совместно используемых сведений, и для обозначения таких сведений стали применять термин data (сначала он употреблялся в более узком значении 'исходные факты для вычислений при решении математических задач').

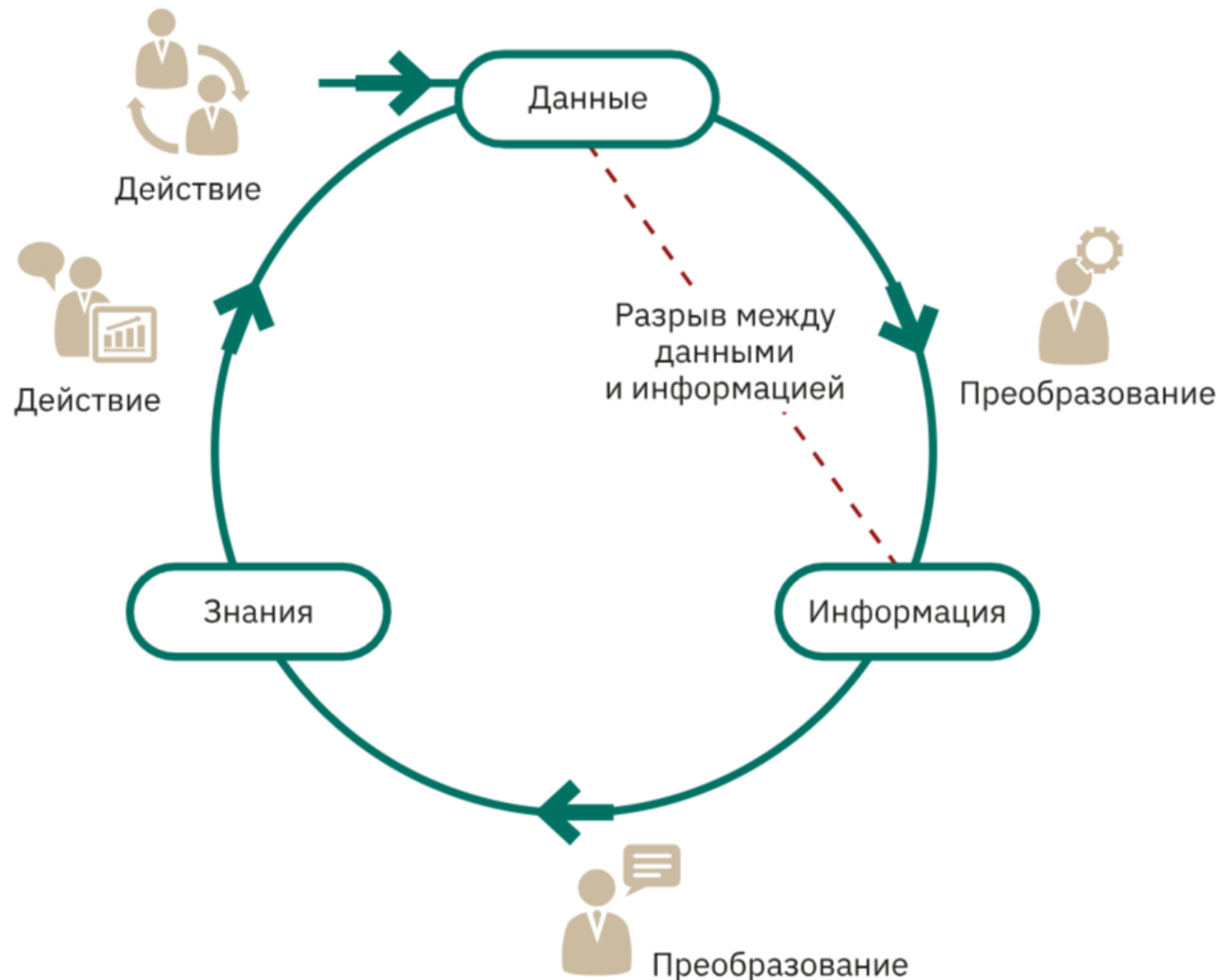


ДАННЫЕ — это дискретные, объективные факты или наблюдения, неорганизованные и необработанные, не передающие никакого конкретного смысла и не имеющие ценности, потому что они лишены контекста и интерпретации.

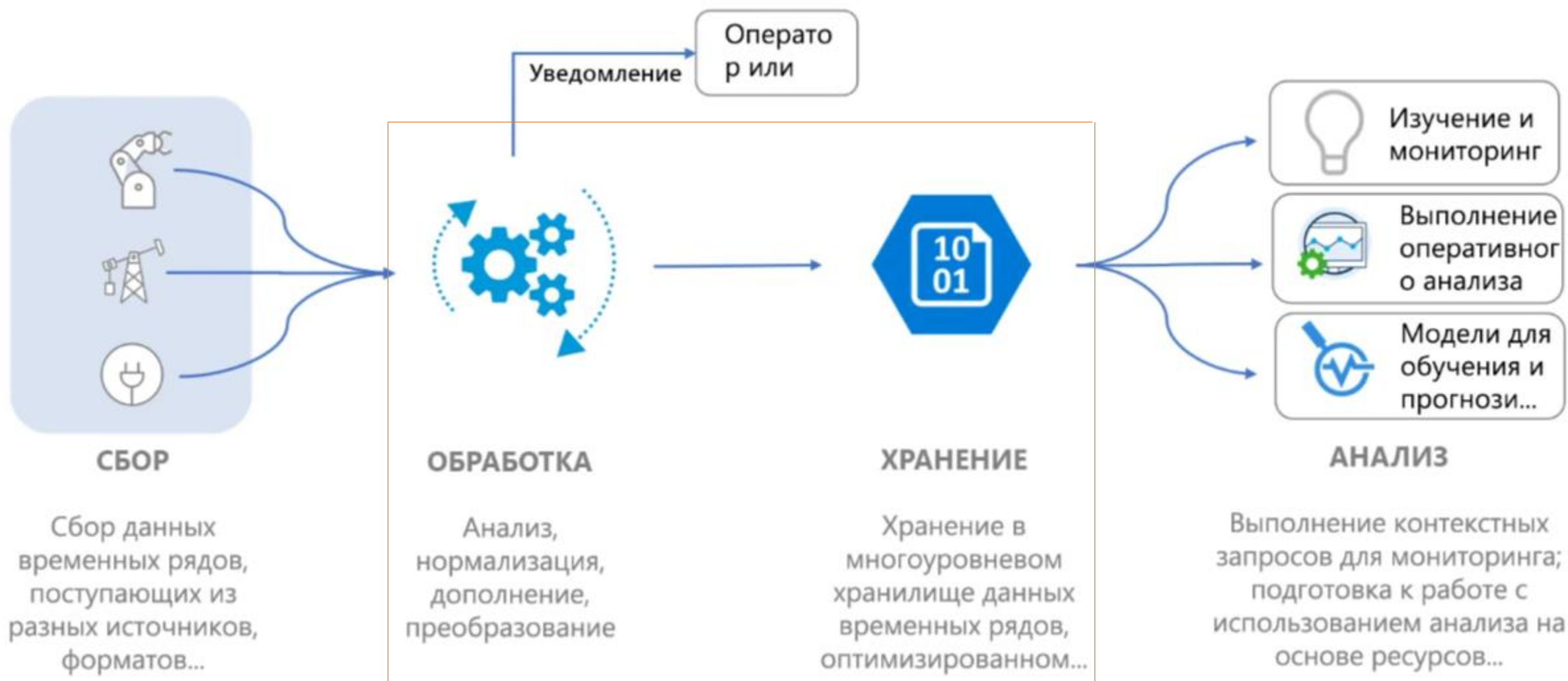
ИНФОРМАЦИЯ — форматированные данные, обработанные с определенной целью, которым придан смысл посредством добавления контекста.

ЗНАНИЯ — это совокупность данных и информации (к которым добавляются экспертные мнения, опыт, другие знания), которая была организована и обработана с целью передачи понимания, накопленных результатов обучения и компетенции так, чтобы получился ценный актив, который можно применить в текущей деятельности для принятия решений.

Данные-Информация-Знания

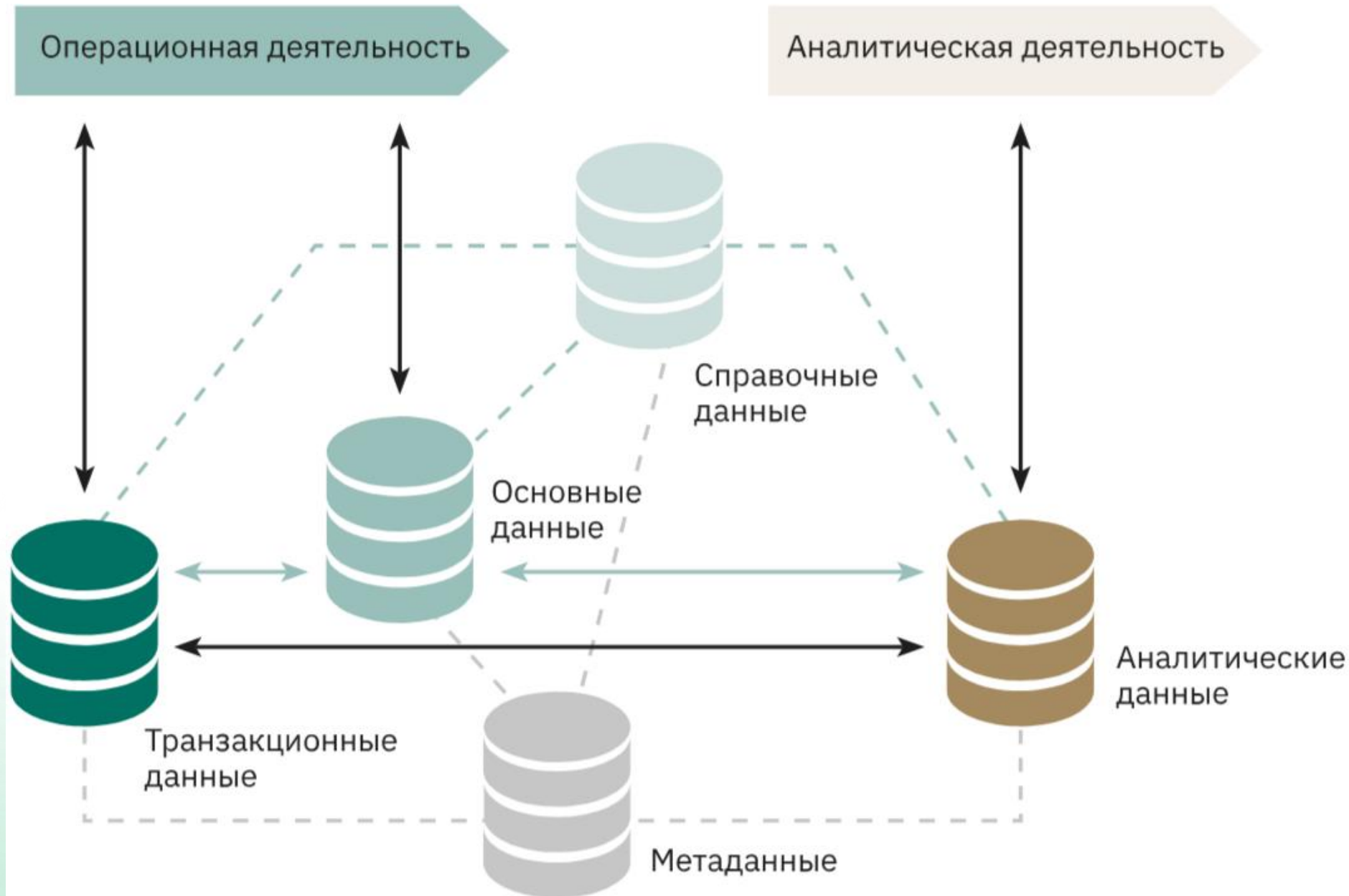


Тема 1_ Базы данных и управление ими



Тема 1_ Базы данных и управление ими

Взаимосвязи основных категорий данных в деятельности



Роли отдельных категорий данных в информационном обеспечении процессов организации

Основные данные (Запись о клиенте)

№ клиента	5001	Тип	03 Коммерческий
Наименование клиента	ООО «Радуга»		
Адрес	Российская Федерация, Алтайский край		
Индекс	659635		

Справочные данные (Тип клиента)

Код	Тип
01	Государственный
02	Муниципальный
03	Коммерческий
04	Сфера образования

Метаданные

Имя поля	Определение	Тип поля	Длина поля
Номер товара	Уникальный номер товара, присвоенный менеджером	Числовое	15
Наименование товара	Краткое наименование товара, указываемое в едином товарном перечне	Символьное	30

Транзакционная запись (Заказ на доставку)

№ заказа	1234	Тип клиента	03 Коммерческий	
№ клиента	5001	Адрес доставки	Российская Федерация, Алтайский край	
Наименование клиента	ООО «Радуга»	Индекс	659635	
Номер товара	Наименование товара	Количество	Цена	Итоговая цена
50223	Сумка для ноутбука	4	1000 р.	4000 р.

Основные данные (Запись о товаре)

Номер товара	50223
Наименование товара	Сумка для ноутбука

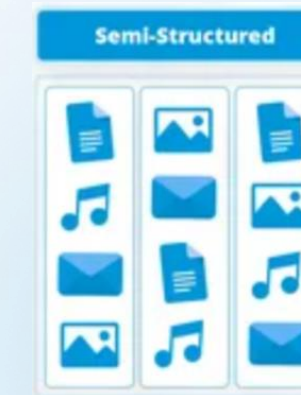
Справочные данные (Цена)

Номер товара	Тип клиента	Цена
50223	01	950
50223	02	950
50223	03	1000
50223	04	900

По степени структурированности можно выделить:

- **структурированные данные** — данные, имеющие строго фиксированную структуру, определяемую формальной моделью данных (например, реляционной схемой);
- **полуструктурированные (слабоструктурированные) данные** — данные, не имеющие строго определенной структуры, но предполагающие наличие правил, позволяющих выделять отдельные семантические элементы при их интерпретации, прежде всего правил расстановки тегов и других маркеров, отмечающих и выделяющих элементы данных (например, файлы, созданные с использованием языка XML и его многочисленных производных, html-страницы и др.);
- **неструктурированные данные** — данные, произвольные по форме, не имеющие строго определенной структуры и не организованные по определенным правилам.

Structured		
1001 1010 1110	1001 1110 0110	0101 1110 1000
1001 1010 1000	1001 0010 1110	1001 1010 1110
1001 1010 1110	1001 1010 1000	0101 1110 1000

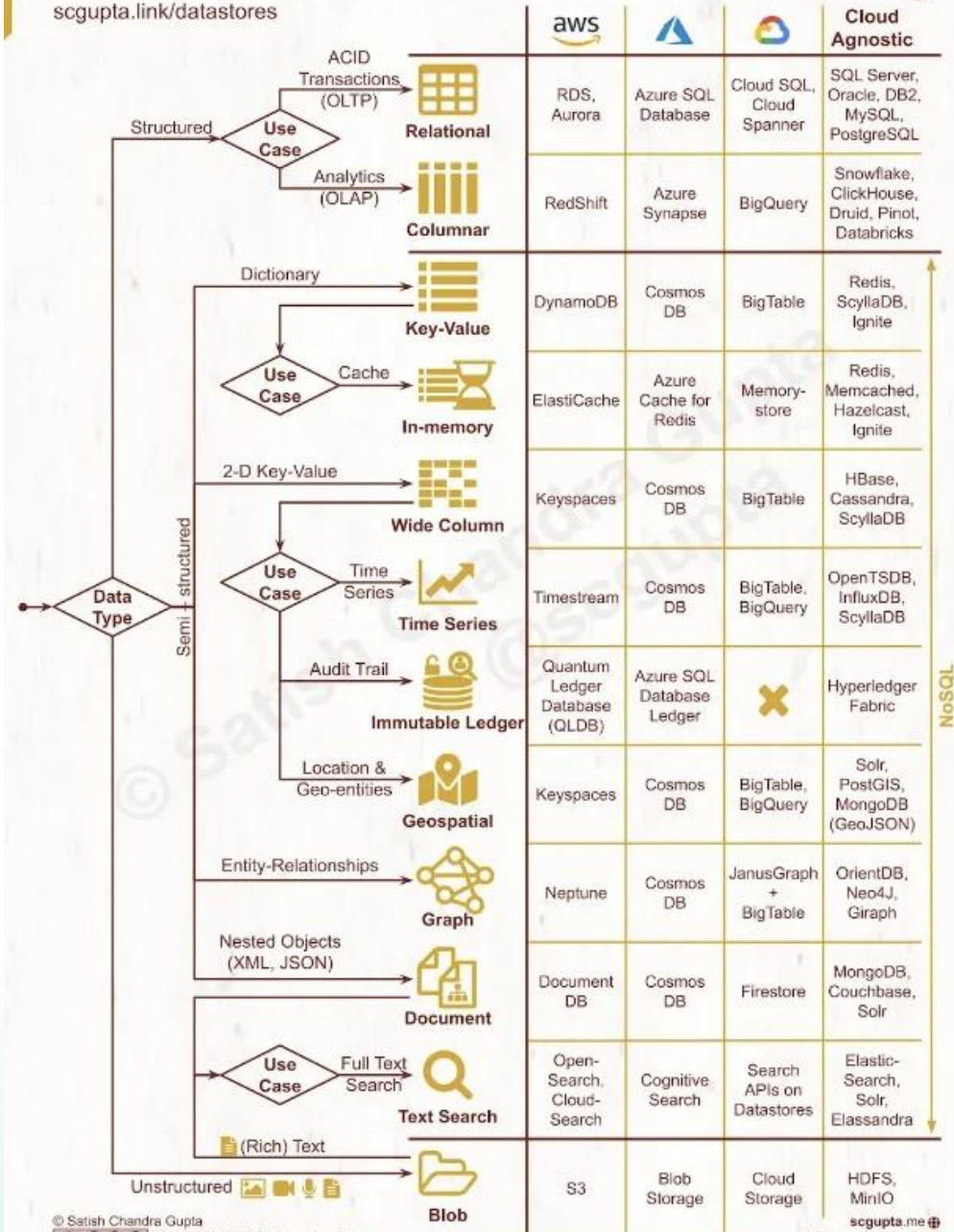


Форматы хранения и передачи данных с разной степенью структурированности



SQL vs. NoSQL: Cheatsheet for AWS, Azure, and Google Cloud

scgupta.link/datastores



© Satish Chandra Gupta

CC BY-NC-ND 4.0 International License
creativecommons.org/licenses/by-nc-nd/4.0/

scgupta.me
 twitter.com/scgupta
 linkedin.com/in/scgupta

Данные и структуры данных

Структурированные и неструктурированные данные (пример)

Структурирование данных

Неструктурированные данные

- Личное дело № 16943, Сергеев Петр Михайлович, дата рождения 1 января 1976г.;
- Личное дело № 16593 Петрова Анна Владимировна, дата рождения 15.03.75
- Личное дело №16693 Анохин Андрей Борисович, дата рождения 14.04.76

Структурированные данные

№ личного дела	Фамилия	Имя	Отчество	Дата рождения
16943	Сергеев	Петр	Михайлови	1.01.75
16593	Петрова	Анна	Владимиرو вна	15.03.75
16693	Анохин	Андрей	Борисович	14.04.76

Подход на основе файлов

Данные хранились в плоских файлах (flat files). Структура: записи, разделенные на поля фиксированной длины.

Проблемы:

- Трудно организовать совместную обработку данных из разных файлов.
- Дублирование данных (это затраты времени на повторный ввод и обработку данных, расход дисковой памяти на их хранение, рассогласование данных, которые в одном месте могли быть изменены, а в другом – нет).
- Зависимость приложений от данных (физическая структура и методы доступа к данным определялись непосредственно в программном коде, поэтому при изменении структуры данных необходимо откорректировать все модули приложения, в которых эти данные используются).
- Невозможность выполнения разовых (ad hoc) запросов, которые изначально не были предусмотрены.
- Нет гарантий защищенности и целостности.
- Проблемы с восстановлением данных после сбоя.
- Сложность организации совместного доступа к данным.
- Нет никакого контроля за доступом к данным, кроме того, который налагается прикладными программами.

- **База данных** – коллекция логически связанных данных и их описаний, предназначенных для удовлетворения информационных потребностей предприятия (организации).
- **База данных** — это некоторый набор перманентных (постоянно хранимых) данных, используемых прикладными программными системами какого-либо предприятия.
- **Описания данных** – системный каталог, или словарь данных, или метаданные (данные о данных).
- **Система управления базами данных (СУБД)** – программная система, позволяющая пользователям определять, создавать, поддерживать данные и управлять доступом к базе данных.
- **Система баз данных** — это компьютеризированная система хранения записей, т. е. компьютеризированная система, основное назначение которой — хранить информацию, предоставляя пользователям средства ее извлечения и модификации.

Главные компоненты системы: данные, аппаратное обеспечение, программное обеспечение, процедуры и пользователи.

- **Данные**

Данные в БД являются интегрированными и разделяемыми.

Интегрированные – объединяют различные данные, исключается избыточность хранения данных.

Разделяемые – обеспечивается совместный доступ к данным (возможно, параллельный).

- **Аппаратное обеспечение**

- **Программное обеспечение**

- сервер базы данных (database server) или система управления базами данных, СУБД (Database Management System — DBMS)
- утилиты
- средства разработки приложений
- средства проектирования
- генераторы отчетов

- **Процедуры**

- вход в систему
- запуск и останов сервера баз данных
- выполнение резервного копирования базы данных
- обработка сбоев и т. д.

- **Пользователи**

- прикладные программисты
- конечные пользователи
- администраторы баз данных (АБД) (database administrators (DBA))
- администраторы данных (АД) (data administrators (DA))
- проектировщики базы данных

- **Администратор данных**

- администратор должен разбираться в данных
- понимать нужды предприятия по отношению к данным *на уровне высшего управляющего звена* в руководстве предприятием

- **Администратор базы данных, или АБД**

Администратор базы данных, в отличие от администратора данных, должен быть профессиональным специалистом в области информационных технологий. Это *технический* специалист, ответственный за реализацию решений администратора данных. Он отвечает за физическую реализацию БД, обеспечение целостности, защищенности, производительности.

Преимущества баз данных

- Возможность совместного доступа к данным
- Сокращение избыточности данных (допустима контролируемая избыточность)
- Обеспечение согласованности данных (Data consistency), т. е. устранение противоречивости данных (до некоторой степени) (это следствие предыдущего пункта)
- Возможность поддержки транзакций при параллельной работе разных пользователей и программ

Транзакция (transaction) — это логическая единица работы.

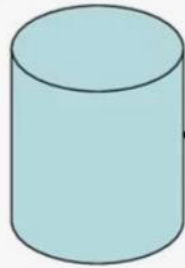
- Обеспечение целостности данных
- Организация защиты данных
- Возможность введения стандартизации
- Обеспечение независимости от данных (data independency)
- Экономия на масштабе за счет централизации данных
- Улучшенный доступ к данным (возможность писать ad hoc запросы)
- Более надежные процедуры резервного копирования данных и восстановления после сбоев

Транзакции

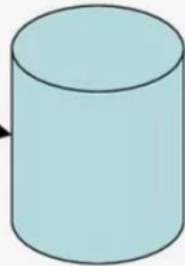
Транзакция – неделимая с точки зрения воздействия на БД последовательность операторов манипулирования данными, такая, что:

- 1) либо результаты всех операторов, входящих в транзакцию, отображаются в БД;
- 2) либо воздействие всех этих операторов полностью отсутствует.

Исходное состояние

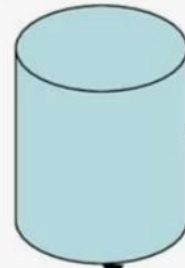


COMMIT



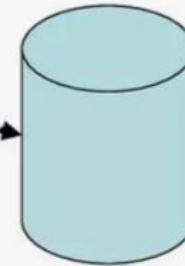
Измененная БД

Исходное состояние



Исходное состояние

ROLLBACK

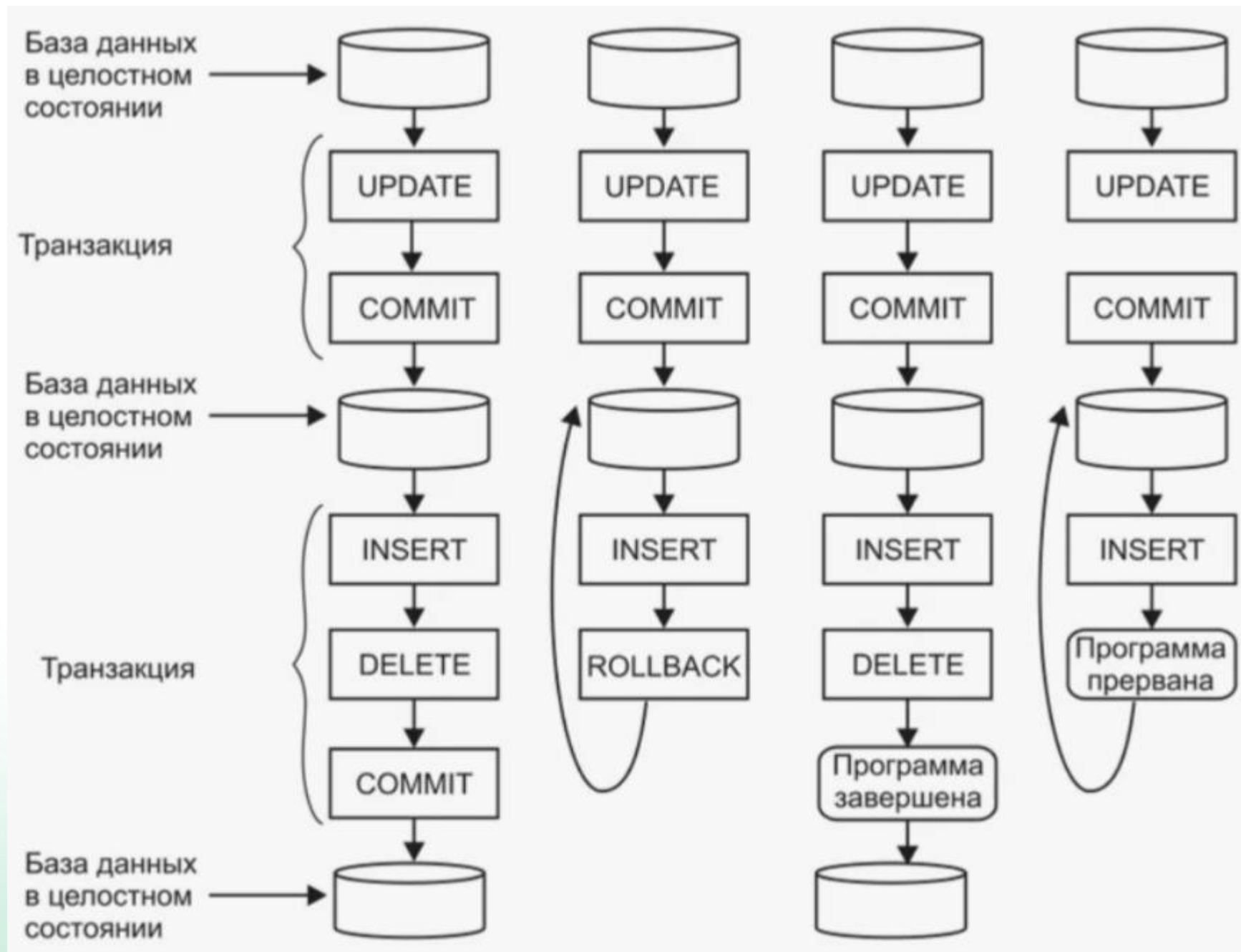


Нарушение целостности

Commit –
транзакционная
команда для
сохранения
изменений

ROLLBACK –
отмена изменений,
сделанных во время
транзакции

Транзакции



Почему необходимо использовать базы данных

Преимущества использования баз данных по сравнению с традиционным бумажным методом содержания информации очевидны. Главным образом они сводятся к следующему:

1. **Компактность.** Нет необходимости в больших помещениях, хранилищах, архивах.
2. **Скорость.** Компьютер обрабатывает данные практически мгновенно (по сравнению с человеком). Это означает, что все операции поиска, изменения, группировки данных и формирования отчетов будут происходить во много раз быстрее, чем при традиционной технологии работы с информацией на бумажных носителях.
3. **Низкие трудозатраты.** Исключается утомительная работа по перебору бумажных документов и выборки необходимой информации из них. Сокращение этой механической работы высвобождает большое количество времени на решение других задач.
4. **Оперативность.** Время между запросом необходимой информации и ее получением снижается в десятки раз, таким образом можно говорить о более быстром, по сравнению с традиционной бумажной технологией, получении последней информации.
5. **Возможность неоднократного использования информации.** Информация, введенная одним пользователем, становится доступной для других. Эта информация может быть использована неограниченное число раз различными пользователями для решения своих задач.

Недостатки баз данных

- Сложность (нужны специалисты)
- Большой масштаб (следствие – требуется мощное аппаратное обеспечение, много памяти, быстрые процессоры)
- Высокая стоимость коммерческих СУБД (но есть альтернатива – свободное ПО, например, PostgreSQL)
- Высокая стоимость перехода от устаревших систем, основанных на файлах, к современным СУБД
- Более низкая скорость работы, чем у специализированных приложений, написанных с учетом специфики решаемой задачи
- Большой масштаб проблем в случае сбоя сервера баз данных

Системы баз данных. Архитектура ANSI/SPARC (1)

- В 1971 г. подразделением DBTG (Data Base Task Group) организации CODASYL (Conference on Data Systems Language) был предложен двухуровневый подход: системное представление (system view), называемое *схемой* и пользовательские представления (user views), называемые *подсхемами*.
- Комитет под названием Standards Planning and Requirements Committee (SPARC) Американского национального института стандартов (The American National Standards Institute (ANSI)), т. е. ANSI/X3/SPARC, в 1975 г. предложил трехуровневую архитектуру.
- Хотя архитектура ANSI/SPARC не стала стандартом, она является базисом для понимания функциональности СУБД.
- Архитектура ANSI/SPARC включает три уровня: внутренний, концептуальный, внешний.

Внешний уровень (*пользовательский логический*) наиболее близок к пользователям, т. е. связан со способами представления данных для отдельных пользователей.

- Одни и те же данные могут быть представлены разным пользователям по-разному.

Концептуальный уровень (называемый также *общим логическим* или просто *логическим*, без дополнительного определения) является «промежуточным» уровнем между двумя первыми. Он описывает, какие данные хранятся в базе данных.

- Если внешний уровень связан с *индивидуальными* представлениями пользователей, то концептуальный уровень связан с *обобщенным* представлением пользователей. Концептуальное представление — это представление всего содержимого базы данных. Оно представляет:
 - все сущности, их атрибуты и связи между ними
 - ограничения, накладываемые на данные
 - смысловую информацию о данных
 - информацию о защищенности данных и их целостности

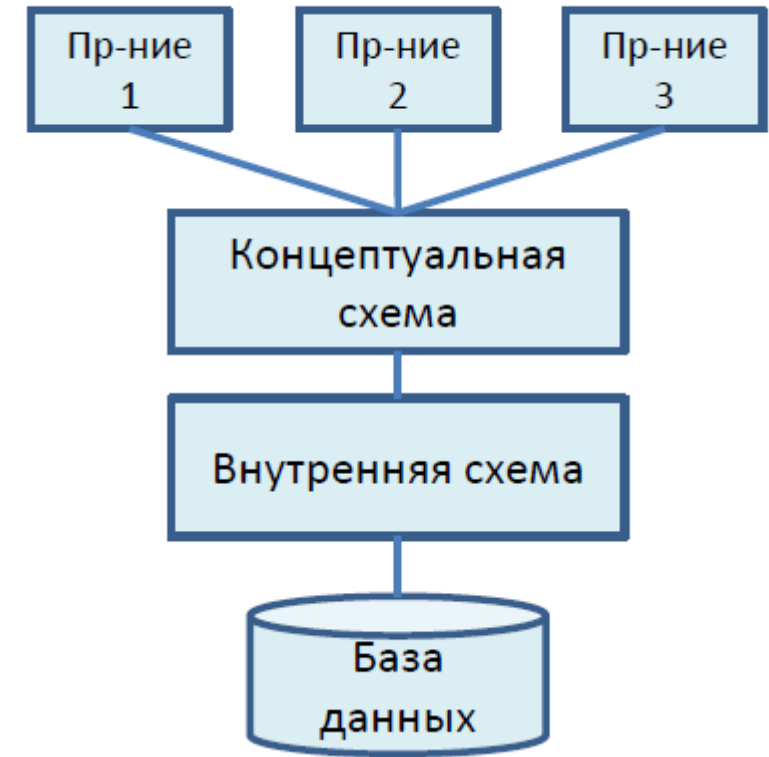
Внутренний уровень (иногда называемый также *физическим*. См. ниже) наиболее близок к физическому хранилищу информации, т. е. связан со способами сохранения информации на физических устройствах. Он описывает, каким образом данные хранятся в базе данных.

- Внутренний уровень отвечает за следующие вещи:
 - выделение пространства для хранения данных и индексов
 - описания хранимых записей (с указанием размеров хранимых элементов данных)
 - размещение записей
 - методы сжатия и шифрования данных

Ниже внутреннего уровня лежит **физический уровень**. Он может реализовываться операционной системой при участии СУБД.

Архитектура ANSI/SPARC

Представления пользователей



Схемы, отображения, экземпляры

- Общее описание базы данных – **схема** базы данных (intension).
- Совокупность данных в базе данных в конкретный момент времени – **экземпляр** базы данных (extension).
- В соответствии с уровнями абстракции:
 - внешние схемы (подсхемы). Их может быть много (больше одной)
 - концептуальная схема
 - внутренняя схема
- СУБД отвечает за отображение между этими схемами:
 - «концептуальная/внутренняя»
 - «внешняя/концептуальная»

Независимость от данных

- Приложения **зависимы** от данных, если сведения об организации данных и способе доступа к ним встроены в саму логику и программный код приложения.
- **Независимость от данных** можно определить как невосприимчивость приложений к изменениям в физическом представлении данных и в методах доступа к ним.
- **Хранимое поле** — это наименьшая единица хранимых данных.
Следует различать тип поля и экземпляр поля.
- **Хранимая запись** — это набор взаимосвязанных хранимых полей.
Следует различать тип записи и экземпляр записи.
- **Хранимый файл** — это набор всех существующих в настоящий момент экземпляров хранимых записей одного и того же типа.
- В базах данных *логическая* (с точки зрения разработчика приложения) запись не всегда совпадает с соответствующей *хранимой* записью.
- База данных должна иметь возможность **расти** и развиваться, не нарушая логическую структуру существующих приложений. Например, добавленные новые хранимые поля будут невидимы для существующих приложений.
- Главная цель трехуровневой архитектуры – обеспечение независимости от данных.

Данные и модели данных

- **Модель данных** — интегрированная совокупность концепций для описания данных (и манипулирования ими), взаимосвязей между данными и ограничений на данные, имеющих место в организации.
- Модель данных включает три компонента:
 - Структурная часть — правила конструирования базы данных
 - Манипуляционная часть — определяет типы операций, которые допустимо выполнять над данными
 - Множество ограничений целостности — требования, предъявляемые к данным и их соотношениям
- Все модели данных можно разделить на три группы:
 - объектные (object-based)
 - основанные на записях (record-based)
 - физические (physical) модели
- **Физические модели данных**
 - unifying model
 - frame memory

Объектные модели данных

Эти модели основаны на следующих понятиях:

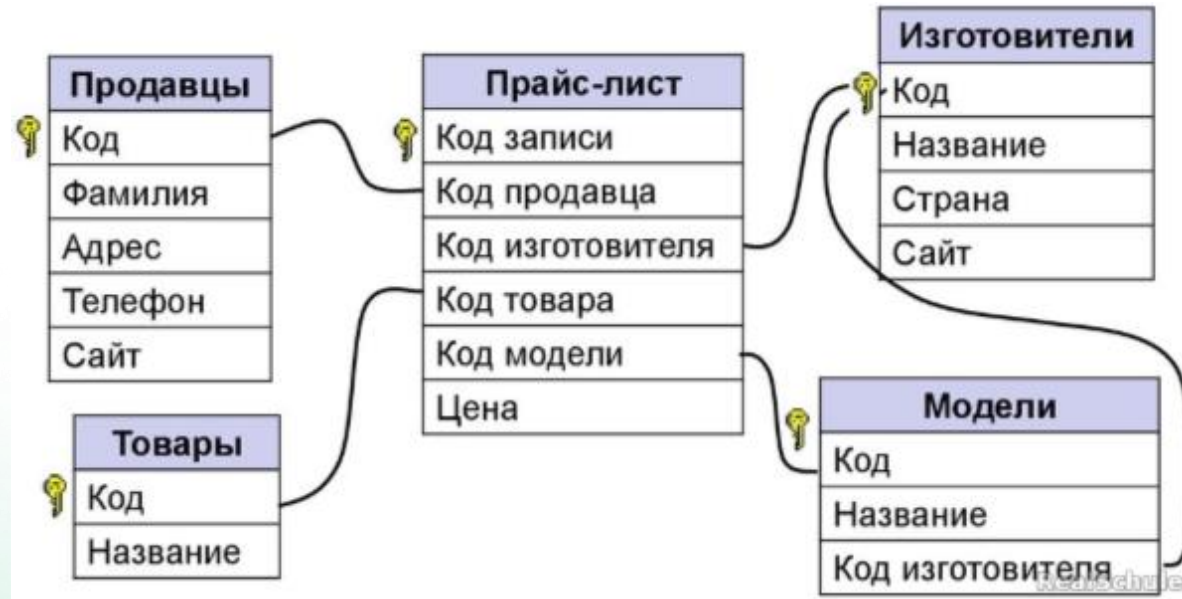
- **Сущность** – любой объект, информация о котором может (должна быть представлена в базе данных)
- **Связь** – соединяет две или более сущностей и указывает вид взаимодействия между ними. Связи также должны быть представлены в базе данных. Связь можно понимать как сущность особого типа.
- **Свойства** – описывают (детализируют сущности и связи).

Виды моделей:

- Entity-Relationship (ER-модель)
- Семантические
- Функциональные
- Объектно-ориентированные.

Модели данных, основанные на записях

- В таких моделях база данных состоит из некоторого количества записей фиксированного формата, возможно, различных типов.
- Каждый тип записей определяет фиксированное число полей, возможно, фиксированной длины.
- Три главных типа таких моделей:
 - реляционная (relational data model)
 - сетевая (network data model)
 - иерархическая (hierarchical data model)

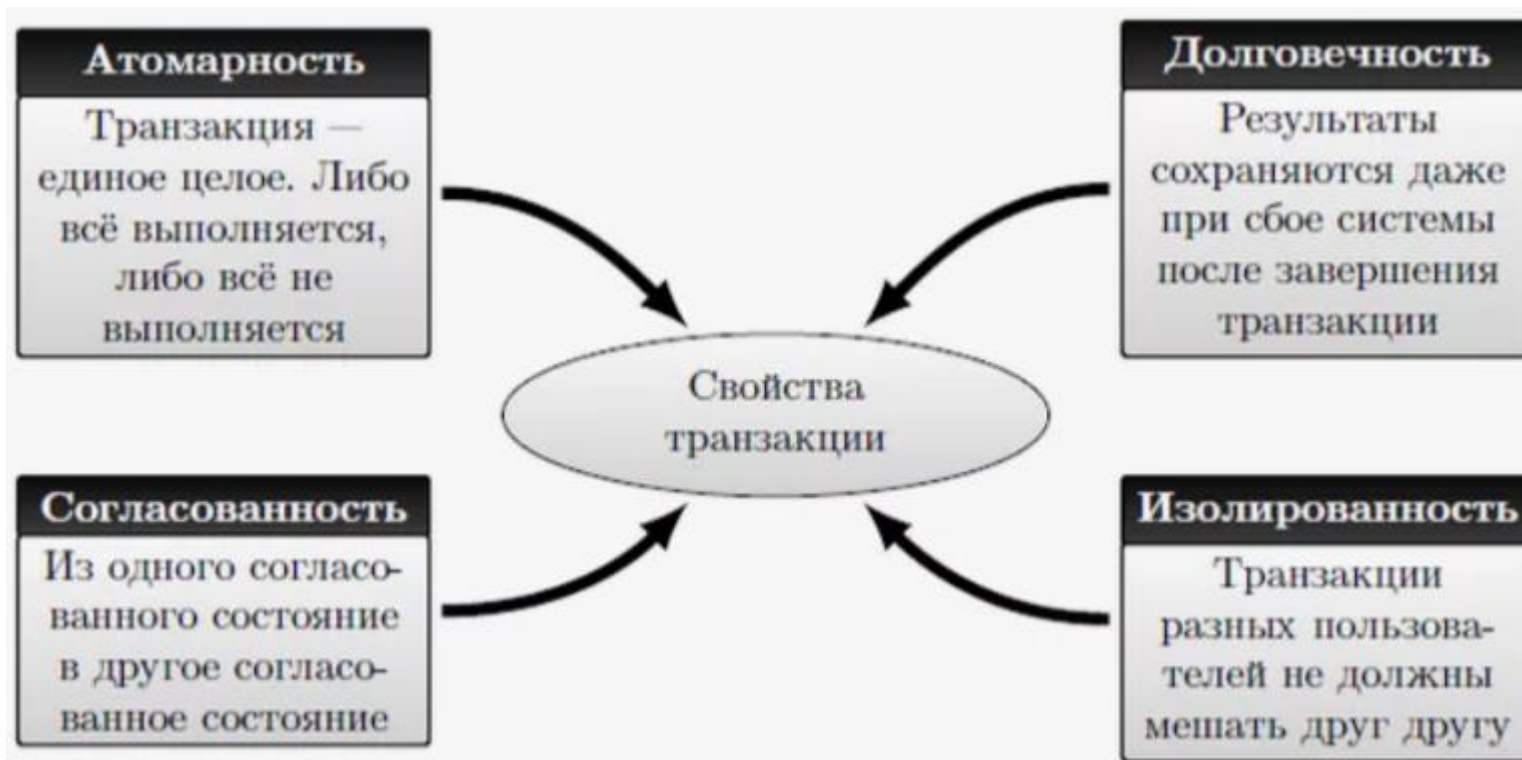


Функции системы управления базой данных (1)

- Хранение, извлечение и обновление данных
- Поддержание словаря данных (системного каталога)
 - имена и типы элементов данных
 - имена таблиц
 - ограничения целостности, накладываемые на данные
 - имена авторизованных пользователей СУБД и их права доступа к объектам базы данных
 - статистика использования объектов базы данных
- Поддержка транзакций
 - Транзакция — одно из важнейших понятий теории баз данных.
 - Она означает набор операций над базой данных, рассматриваемых как **единая и неделимая** единица работы, выполняемая полностью или не выполняемая вовсе, если произошел какой-то сбой в процессе выполнения транзакции.
 - Таким образом, транзакции являются средством обеспечения согласованности данных.

Функции системы управления базой данных (2)

- Поддержка параллельности (concurrency) в использовании базы данных
- Восстановление данных после сбоев
- Авторизация пользователей
- Обеспечение доступа к базе данных с компьютеров локальной или глобальной сети
- Защита и поддержка целостности данных
- Реализация принципа независимости от данных
 - Приложения не должны зависеть от фактической структуры базы данных
- Обслуживание базы данных (с помощью различных утилит)
- Оптимизация и выполнение запросов к базе данных



Если вы работаете с базами данных, важно знать об ACID:

-Атомарность (A): все операции транзакции либо выполняются полностью, либо не выполняются вообще. Промежуточные состояния недопустимы.

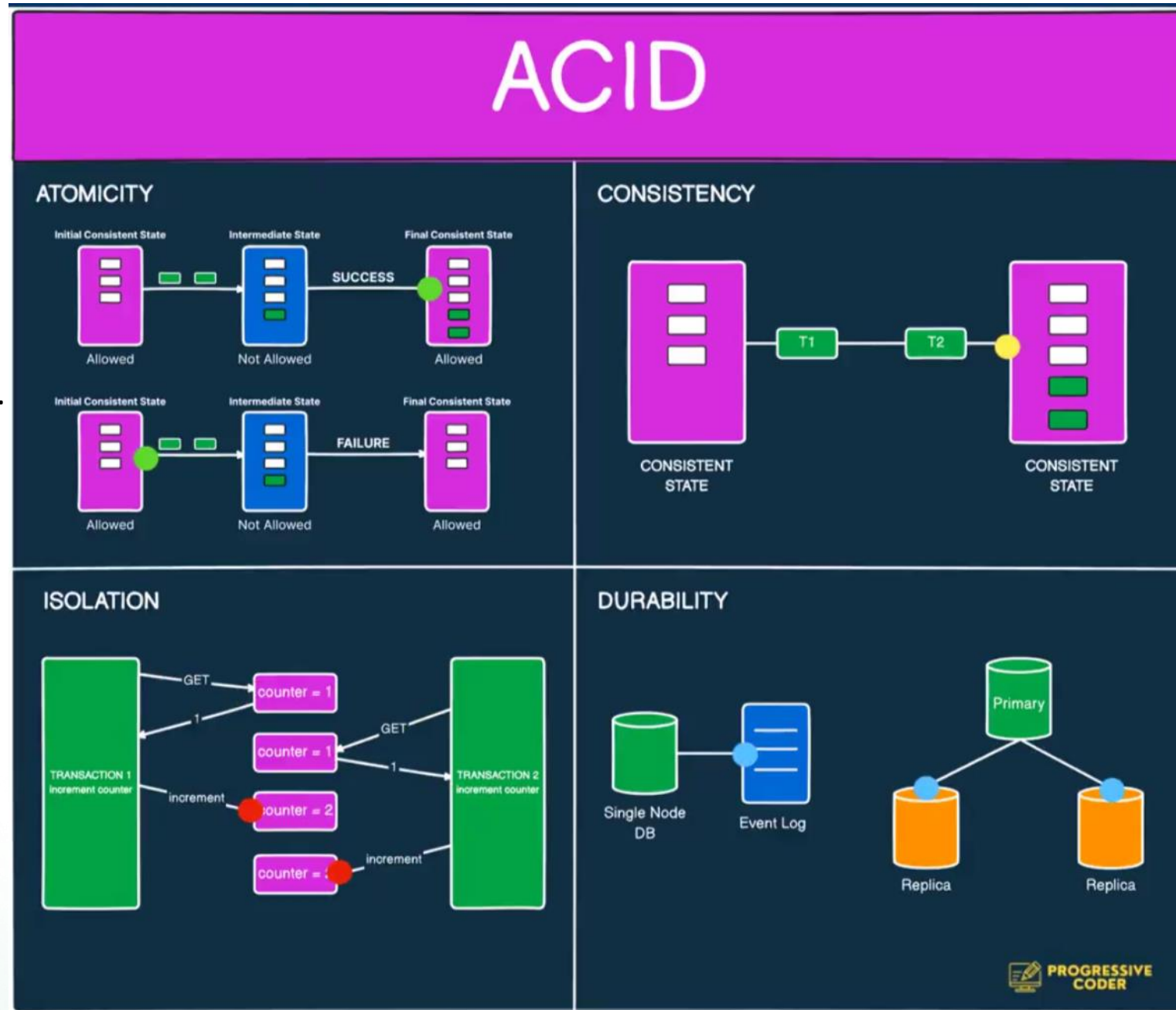
-Согласованность (C): состояние базы данных должно оставаться корректным до и после транзакции. Это зависит от логики приложения.

- Изоляция (I): одновременные транзакции не должны влиять друг на друга. Уровни изоляции варьируются: Read Uncommitted, Read Committed, Repeatable Read, Serializable.

- Долговечность (D): данные сохраняются даже при сбоях системы (аппаратные ошибки, краш базы). Это достигается с помощью журналов или репликации.

ACID гарантирует надежность и согласованность работы с базой данных

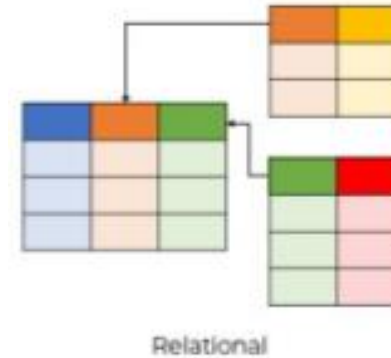
ACID – набор требований к БД



Виды баз данных. Типы СУБД

- Реляционные
- Ключ-значение
- Документно-ориентированные
- Базы данных временных рядов
- Графовые базы данных
- Поисковые базы данных (Search Engines)
- Объектно-ориентированные базы данных
- RDF (Resource Description Framework)
- Wide Column Stores
- Мультимодальные СУБД
- Native XML СУБД
- GEO/GIS (пространственные) и специализированные СУБД
- Event СУБД (баз данных переходов состояний)
- Контентные СУБД
- Навигационные (Navigational) СУБД
- Векторные базы данных

SQL DATABASES



NoSQL DATABASES



Column



Key-Value



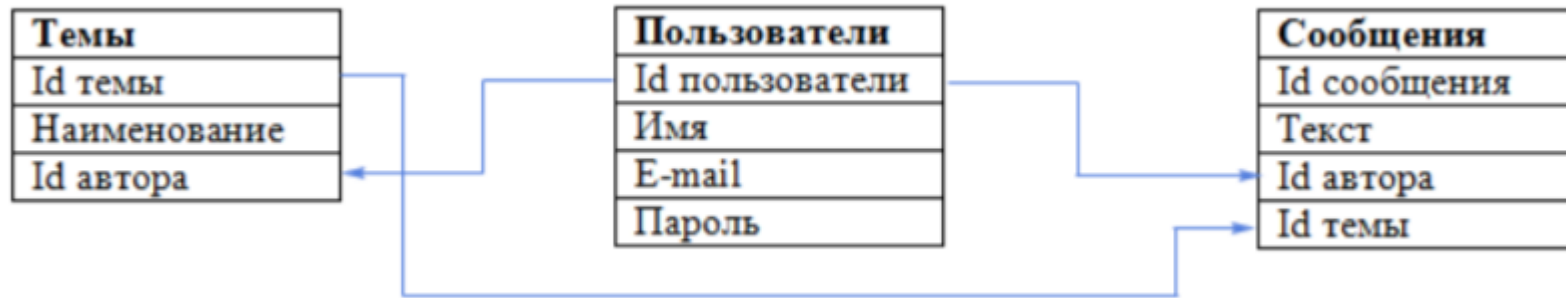
Graph



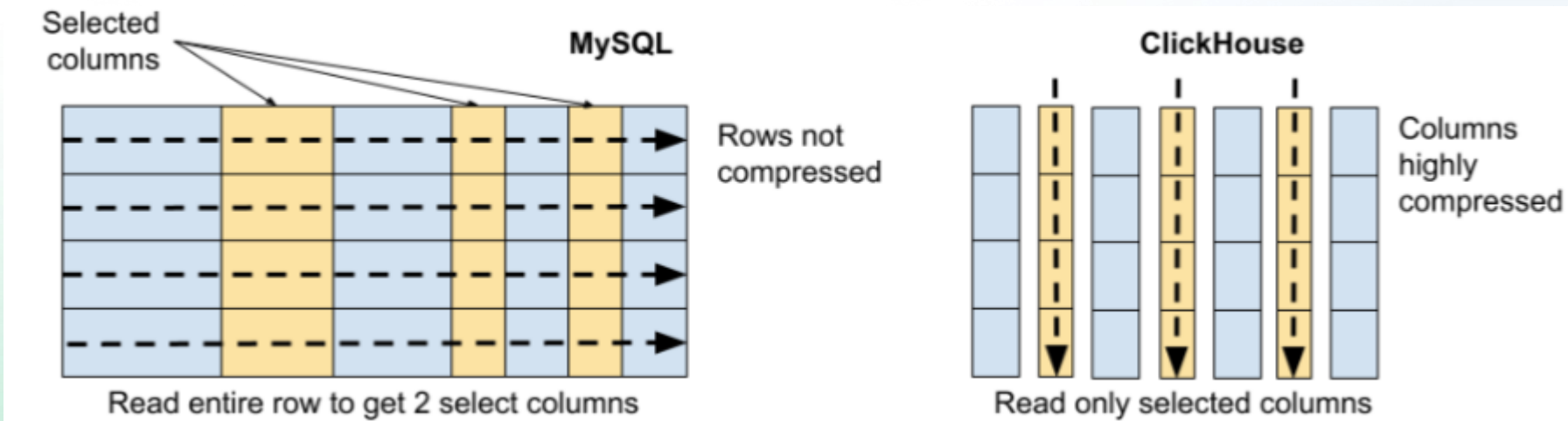
Document

Реляционные базы данных

• Наиболее известными реляционными базами данных являются Open Source проекты PostgreSQL, MySQL и SQLite, а также проприетарные решения Oracle, Microsoft SQL Server и IBM Db2Relational.

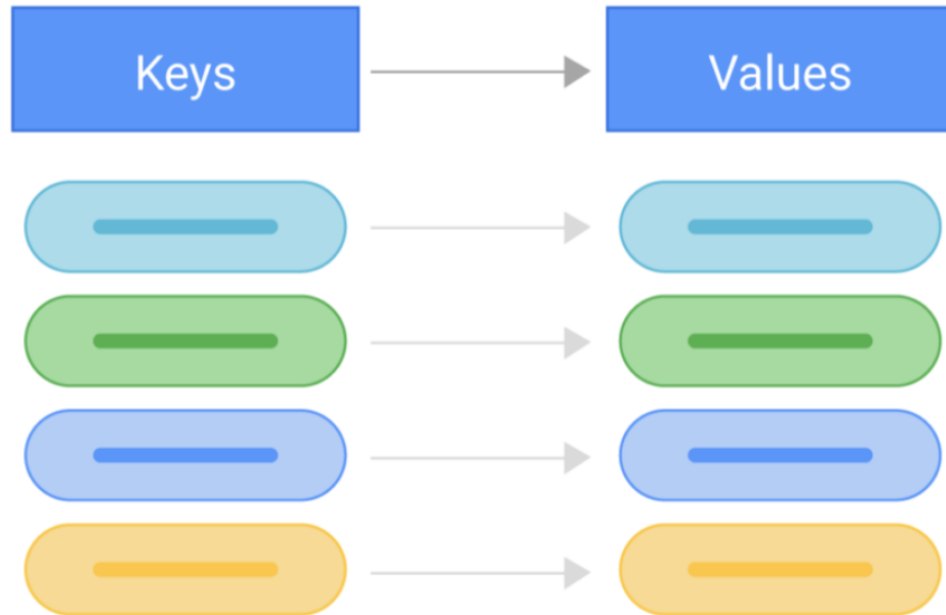


Стоит заметить, что реляционные базы бывают с хранением данных по строкам (PostgreSQL) и по столбцам/колонкам (ClickHouse, Vertica). Колоночные/столбцовые базы лучше подходят для аналитики, в то время как ориентация на строки лучше подходит для транзакционных нагрузок.



Key-value (ключ значение) базы данных

Тип баз данных Key-value предназначен для осуществления быстрых, почти мгновенных запросов для таких задач как кэш, отображение баланса и т.д.. Высокая скорость осуществляется за счет хранения данных по принципу ключ-значение, и в большинстве случаев благодаря работе в оперативной памяти.



Из отечественных СУБД здесь следует отдельно выделить Tarantool, распространяемый под лицензией Упрощенная BSD и имеющий отдельную версию для крупных корпоративных клиентов. В Tarantool реализована гибридная схема данных: key-value, документно-ориентированная, реляционная и пространственная.

Словари содержат коллекцию объектов или записей, а объекты содержат множество различных полей, каждое из которых содержит данные. Записи хранятся и извлекаются с использованием ключа, который однозначно идентифицирует запись и используется для быстрого поиска данных. Основным применением является ускорение отображения данных для конечных пользователей и снижение нагрузок, в том числе I/O на инфраструктуру организаций. Наиболее известными и широко используемыми Key-Value решениями являются Redis и Memcached.

Документо-ориентированные базы данных

Если вам нужно хранить много файлов/документов и вы не хотите задумываться (до разумных пределов, разумеется) о структуре хранения, иерархии, связях, вам может подойти одна из документо-ориентированных баз данных. Дополнительным преимуществом являются широкие возможности для масштабирования.



Документо-ориентированные базы данных созданы для хранения иерархических структур данных (документов). Основой документоориентированных СУБД являются документные хранилища, имеющие структуру дерева или леса. Деревья начинаются с корневого узла и может содержать несколько внутренних и листовых узлов. Листовые узлы содержат данные, которые при добавлении документа заносятся в индексы, это дает возможность даже при достаточно сложной структуре находить путь к искомым данным. В отличие от хранилищ типа ключ-значение, выборка по запросу к документному хранилищу может содержать части большого количества документов без полной загрузки этих документов в оперативную память.

Наиболее популярной документо-ориентированной базой данных на текущий момент является MongoDB..

Документо-ориентированные базы данных

Из отечественных решений, к документо-ориентированным базам данных можно отнести СУБД Енисей. И в рамках одной из модальностей к таким базам можно отнести YDB и YTsaurus.



Базы данных временных рядов (TSDB)

Если у вас есть упорядоченные по времени данные с временными метками, такие как метрики от инфраструктуры или данные датчиков, может быть полезно использовать одну из баз данных временных рядов (Time Series Data Base).

Принцип хранения данных в базе данных временных рядов. Как мы видим, важной особенностью является наличие столбца соответствующего времени события.

Measurement Time	Air Quality Index (AQI)	The density of PM2.5
2018/01/01 00:00	156	45
2018/01/01 01:00	101	29
2018/01/01 02:00	97	19
...	...	...
2018/12/31 21:00	133	34
2018/12/31 22:00	135	36
2018/12/31 23:00	141	43

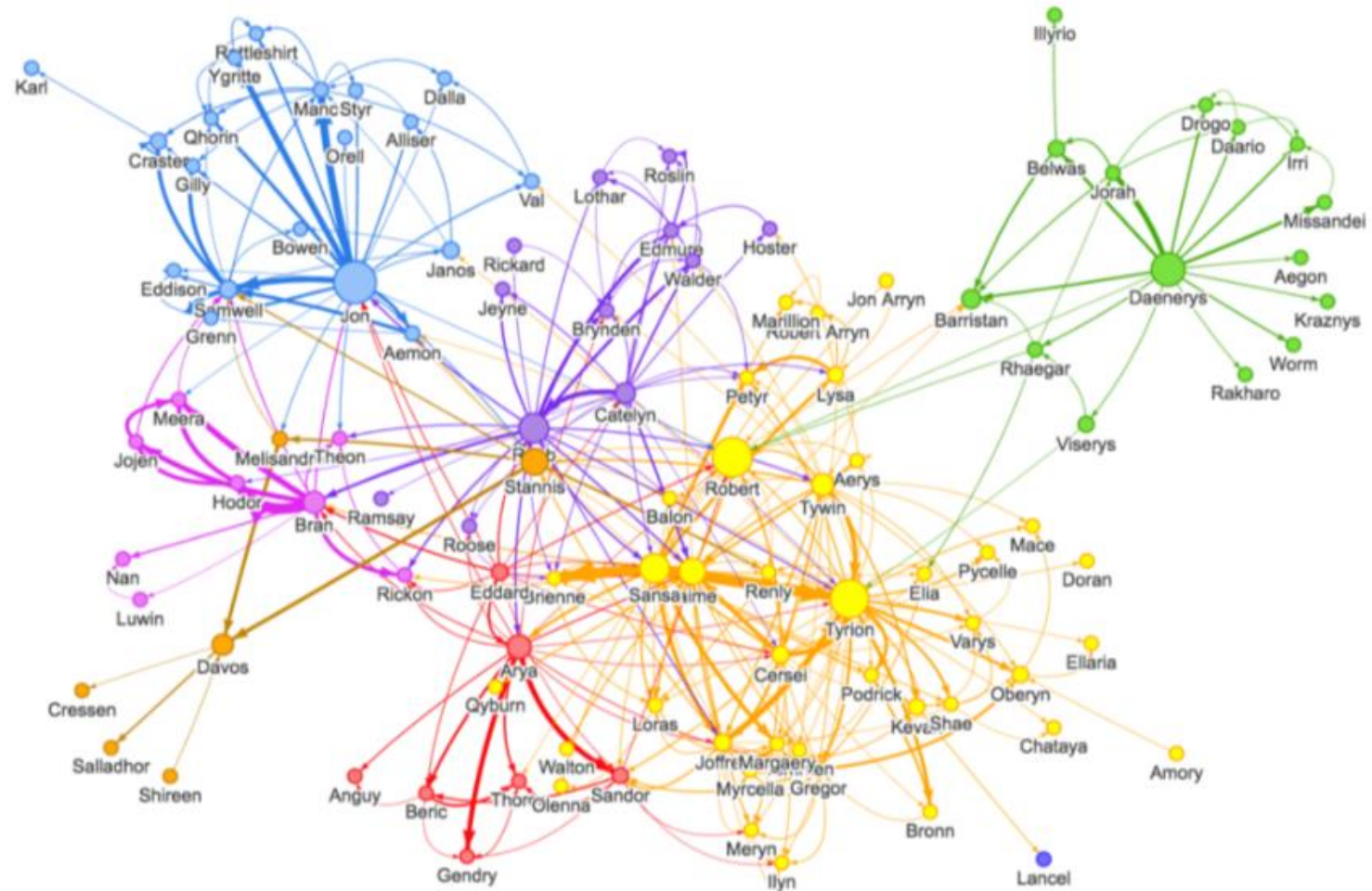
Общие характеристики баз данных временных рядов: Данные временных рядов всегда собираются на протяжении определенного периода времени. Данные из рабочих нагрузок являются новыми и записываются как вставки. Уже существующие данные не обновляются путем замены. Когда данные записываются, они автоматически назначаются последнему интервалу времени. Базы данных временных рядов часто используются для осуществления мониторинга различных метрик (будь то загрузка CPU, или показатели работы какого-либо датчика).

Графовые базы данных

Если вам нужно анализировать отношения данных, их связи или просто упростить запросы с большими Join, имеет смысл использовать графовые базы данных. Данные и их связи представляются как вершины и ребра графа соответственно. Таким способом можно легко представить денежные переводы (для определения различных мошеннических схем), связи в социальной сети и граф общения между операторами сотовой сети.

Чаще всего графовые базы данных используются в финансовой сфере для выявления различных мошеннических схем, но они будут удобны и для любой другой задачи, где нужно работать со связями объектов.

Наиболее известное Open Source решение, это Neo4J, но есть и ряд других качественных альтернатив.



Поисковые базы данных (Search Engines)

Если вам необходимо осуществлять поиск большим объемам данных, особенно неструктурированным, как пример поиск по нескольким терабайтам логов, то вам может пригодиться использовать базу данных, совмещающую с функционалом хранения информации еще и функционал поиска по текстам.

Представим, что у вас есть N петабайт логов (или других текстовых данных). Обычный поиск по словам уже не подойдет, чтобы осуществить поиск и аналитику в разумное время. На помощь приходит индексирование. Если очень утрировано его рассмотреть, можно его представить следующим способом. Каждому слову/лемме/n-грамме присвоим индекс и запишем эти индексы в специальную таблицу, где строки, это документ, а в столбики это индексы. Похожая система используется для построения систем поиска плагиата, правда там чаще применяют не слова, а шинглы (индексы с наложением).

Пример генерации шинглов для дальнейшего использования в поиске плагиата

Искать по индексу существенно быстрее, чем по совпадению по словам в документах. Для поиска по документам можно использовать и эмбединги нейронных сетей, в которых закодирован "смысл" высказываний. Но для данной задачи лучше подойдут векторные базы данных. Разумеется, современные поисковые СУБД предлагают значительно более широкий функционал. Наиболее популярны такие решения как Elasticsearch (и его версия - OpenSearch), проприетарный Splunk.



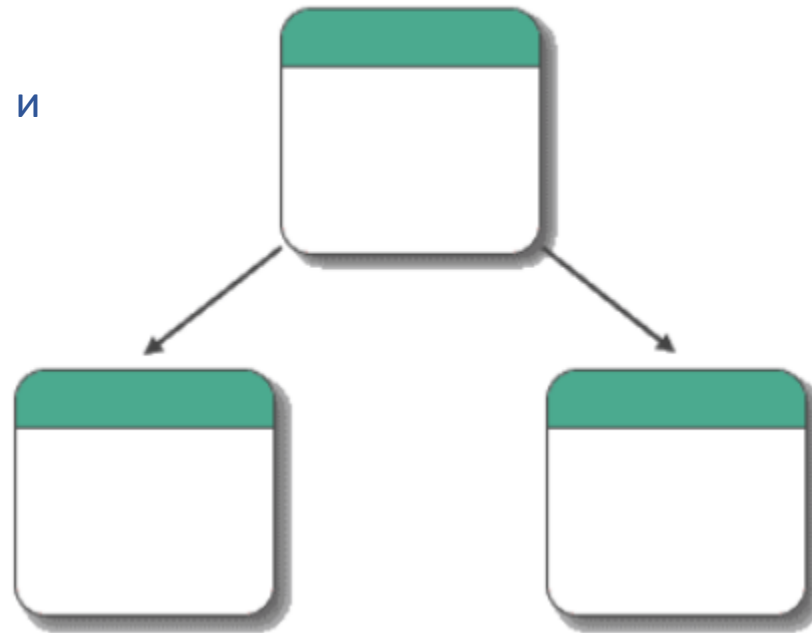
Объектно-ориентированные базы данных

Объектно-ориентированные базы данных представляют собой базы данных, в где информация представлена в виде объектов, как в объектно-ориентированных языках программирования. Объектно-ориентированные базы данных появились как способ нативной коммуникации кода написанного с использованием объектно-ориентированных языков с базой данных.

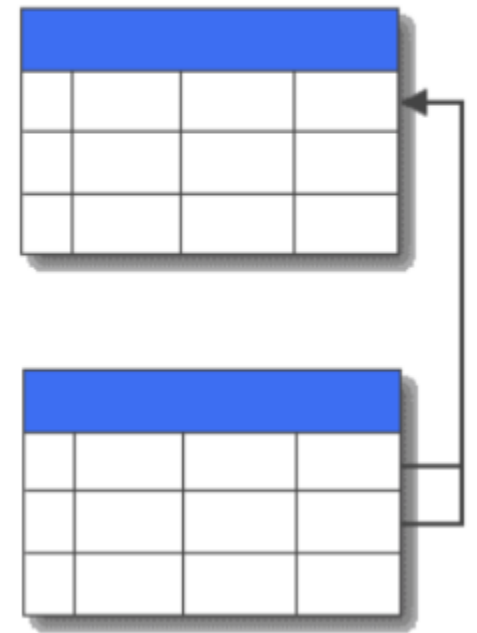
Объектно-ориентированные базы данных обладают следующими преимуществами:

- нет проблемы несоответствия модели данных в бд и приложении, так как в БД они сохраняются в том же виде.
- не нужно отдельно поддерживать модель данных на стороне базы данных.
- все объекты на уровне источника строго типизированы.

Object-Oriented



Relational



Сравнение объектно-ориентированной и
реляционной СУБД

Wide Column Stores

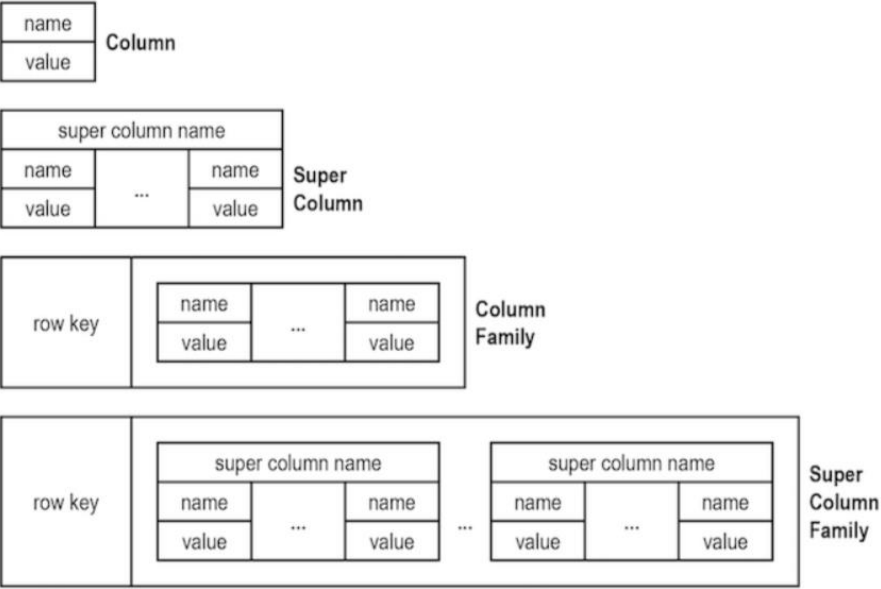
Wide Column Stores (расширяемые хранилища записей) — это база данных NoSQL, в которой хранение данных организовано в виде гибких столбцов, которые можно распределить по нескольким серверам или узлам базы данных, используя многомерное сопоставление для ссылки на данные по столбцам, строкам и отметкам времени.

Преимущества Wide Column Stores баз данных включают скорость выполнения запросов, масштабируемость и гибкую модель данных.

Отличием от реляционных баз данных является следующее.

Система управления реляционными базами данных хранит данные в таблице со строками, которые охватывают несколько столбцов. Если для одной строки требуется дополнительный столбец, этот столбец должен быть добавлен ко всей таблице со значениями NULL или по умолчанию для всех остальных строк. Если вам нужно запросить в этой таблице СУБД значение, которое не проиндексировано, сканирование таблицы для поиска этих значений будет очень медленным.

Wide Column СУБД имеют концепцию строк как и реляционные базы данных, но чтение или запись строки данных состоит из чтения или записи отдельных столбцов. Столбец записывается только в том случае, если для него есть элемент данных. На каждый элемент данных можно ссылаться по ключу строки, запрос значения оптимизирован так же, как запрос индекса в СУБД.

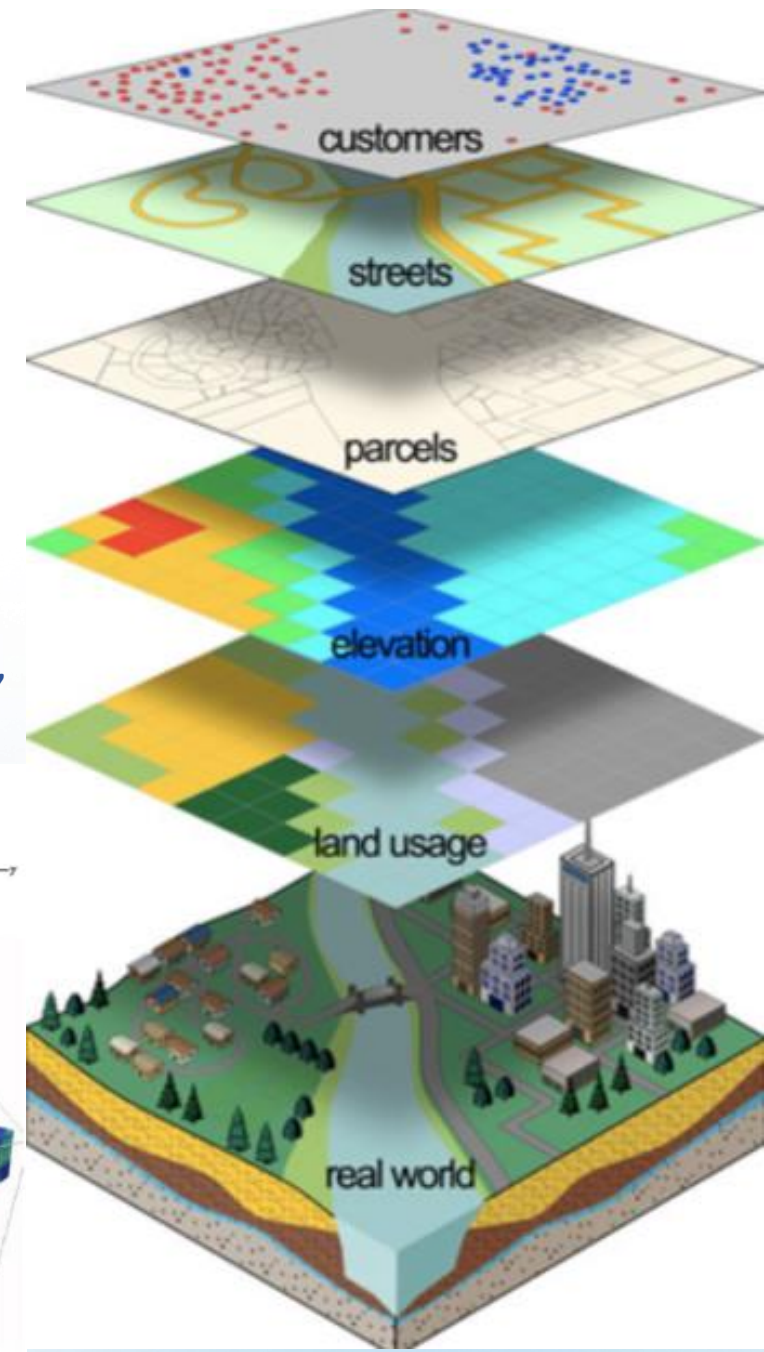
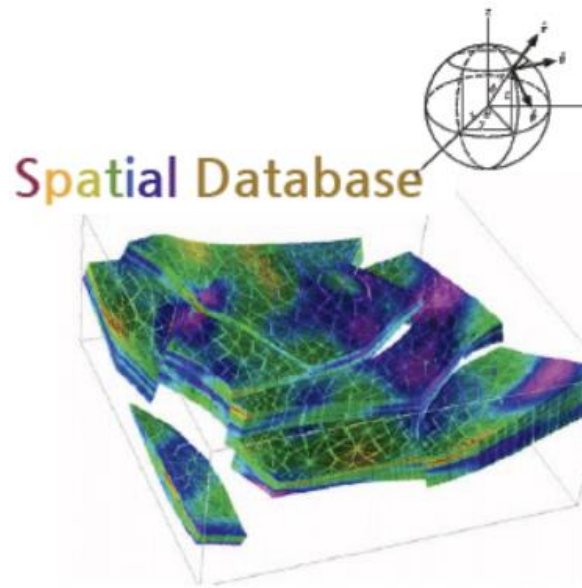


Первая модель столбца - это таблица сущностей/атрибутов/значений. Внутри каждого объекта (столбцы) есть таблица значений/атрибутов. Концепция Wide Column СУБД позволяет строить решения для обработки действительно больших объемов данных. Наиболее известными реализациями являются СУБД Cassandra и Hbase.

GEO/GIS (пространственные) и специализированные СУБД

Если вы создаете картографический сервис, или специализированное приложение использующее данные о локации, будет логично использовать специализированные СУБД под хранение и обработку данных координат.

В отличие от других видов баз данных предназначенных для хранения и обработки числовой и символьной информации, пространственные базы данных обладают возможностями работы с целостными пространственными объектами, объединяющими как традиционные виды данных (описательная часть или атрибутивная), так и геометрические (данные о положении объекта в пространстве). Пространственные базы данных, позволяют выполнять аналитические запросы, содержащие пространственные операторы для анализа пространственно-логических отношений объектов («пересекается...», «касается...», «содержится в...», «содержит...», «находится на заданном расстоянии от...», «совпадает...» и другие).





Types

ByteByteGo

Sec. 10

Relational DB

* Data is stored in tables

Name	Value
Rohit	25
Alex	26
Adam	24
Joe	25
John	40
Arshu	25

Has Pk

Blocked

Name	P
Rohit	3

Name	B
John	2
Arshu	1

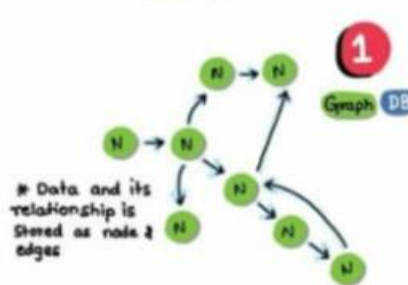
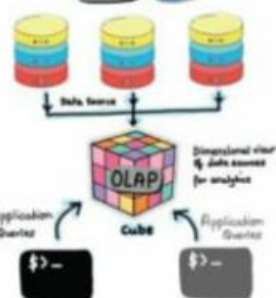
Follows

Name
Alex
Adam
Joe

* Good for storing transactions

* Two table may have a virtual relation in between

OLAP DB



Graph DB

KEY VALUE DB

KEY	VALUE	VALUE
K1	AB	ABAB
K2	B	
K3	CCC	CCCC
K4	D	DDD

* Data is stored like a map of key to values

* Great for fast searches

NoSQL DB



3

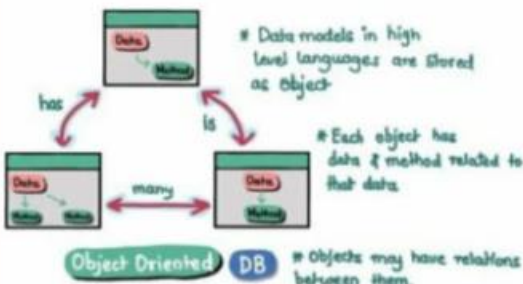
Column Oriented DB

ID	NAME	CITY
4	Rohit	DELHI

* Data is stored in column fashion unlike Relational DB

4

DBs that are non traditional & those don't use SQL Structured Query Language.



ORM: Object Relational mapping - A framework to transform objects from high level language to tables in Relational DB



Most Popular Open Source Databases

Cassandra

- Originally developed by Facebook
- Fault-tolerant distributed NoSQL DB
- High Performance & Scalable

MariaDB

- Fork of MySQL
- Relational database
- Transaction processing workloads

PostgreSQL

- Offers Full RDBMS Features
- ACID Compliant
- SQL query support

Neo4j

- NoSQL Graph Database
- ACID compliant
- Suitable for Knowledge graphs, GenAI apps

SQLite

- Lightweight embedded RDBMS
- Uses disk files
- Great for mobile apps, IOT devices, web browsers

CockroachDB

- Distributed SQL database
- Started as open-source. Now source available
- Scales horizontally & suitable for high-volume OLTP applications

Redis

- NoSQL in-memory database
- Converted to source available in 2024
- Ideal for database query caching layer. Supports pub/sub as well

MongoDB

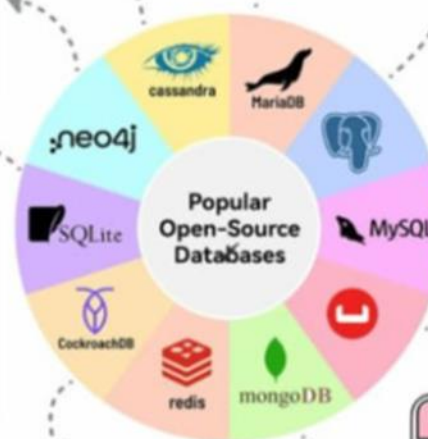
- NoSQL document databases
- Started as open source but moved to source available
- Supports versatile use-cases

Couchbase

- NoSQL database with multimodel capabilities
- Strong consistency, distributed ACID transactions
- Includes vector and full-text search capabilities

MySQL

- Most widely adopted open-source DB
- Relational database for OLTP
- Use for web app servers, cloud apps and enterprise apps



Cloud Database Cheat Sheet

👉 @database_info

DB Type		aws	Azure	Google Cloud	Open Source / 3rd Party
Structured	Relational 	 RDS	 SQL Database	 Cloud SQL	 Oracle  PostgreSQL
	Columnar 	 Redshift	 Synapse Analytics	 BigQuery	 MySQL  SQL Server  Snowflake  Click House
Semi Structured	Key Value 	 DynamoDB	 Cosmos DB	 BigTable	 Redis  Scylla
	In-Memory 	 ElastiCache	 Azure Cache for Redis	 Memory Store	 Redis  Memcached
	Wide Column 	 Keyspaces	 Cosmos DB	 BigTable	 Cassandra  Scylla
	Time Series 	 Timestream	 Time Series Insights	 BigTable	 Influx  OpenTSDB
	Immutable Ledger 	 Quantum Ledger DB	 Confidential Ledger	 CloudSpanner	 Hyper Ledger Fabric
	Geospatial 	 Keyspaces	 Cosmos DB	 BigTable	 PostGIS  geomesa
	Graph 	 Neptune	 Cosmos DB	 CloudSpanner	 OrientDB  Dgraph
	Document 	 Document DB	 Cosmos DB	 FireStore	 MongoDB  Couchbase
UnStructured	Text Search 	 OpenSearch	 Cognitive Search	 CloudSearch	 Elastic search  Elassandra
	Blob 	 S3	 Blob Storage	 Cloud Storage	 Ceph  OpenIO

Структурированные базы данных 📌

Структурированные базы данных организуют данные в predetermined схемы и модели.

Реляционные базы данных, такие как MySQL и PostgreSQL, хранят данные в таблицах со строками и столбцами.

Колоночные базы данных, такие как Amazon Redshift и Google BigQuery, также имеют структурированную модель данных, но хранят их по-другому, оптимизируя для аналитических запросов.

Преимущества:

- Эффективные SQL-запросы
 - Возможность применения ограничений и валидации
 - Последовательность там, где это необходимо
- Примеры использования: CRM-системы, управление запасами, бухгалтерский учет, аналитика

Неструктурированные базы данных 📌

Неструктурированные базы данных оптимизированы для хранения и обработки огромных объемов разнородных данных, таких как документы, изображения, видео. Примеры: AWS S3, Azure Blob Storage.

Преимущества:

- Хранение огромных объемов данных
- Высокая масштабируемость

Примеры использования: Медиарепозитории, управление контентом, океаны данных, журнальные данные, резервное копирование.

Полуструктурированные базы данных 📌

Полуструктурированные базы данных обеспечивают гибкость, храня данные без соблюдения формальной схемы. Данные часто хранятся в виде JSON или других гибких форматов.

Примеры включают в себя документные базы данных, такие как MongoDB, графовые базы данных, наподобие Neptune, широкие колоночные хранилища, такие как ScyllaDB, и хранилища ключ-значение, такие как DynamoDB.

Преимущества:

- Гибкость для изменяющихся данных
- Масштабируемость на разных серверах

Примеры использования: Электронная коммерция, ленты социальных сетей, данные IoT

10 структур данных, которые делают базы данных быстрыми и масштабируемыми:

● **Хеш-индексы:** Обеспечивают доступ к данным за $O(1)$, сопоставляя ключи напрямую с ячейками памяти, что ускоряет точечные запросы. Идеально подходят для кэширования и баз данных в памяти.

● **В-деревья:** Организуют данные в сбалансированных древовидных структурах, обеспечивая эффективное добавление, удаление и обработку диапазонных запросов.

● **Список с пропусками:** Использует слоистые связанные списки для быстрого поиска, добавления и удаления данных без строгих требований к балансировке.

● **Memtable:** Хранит недавние операции записи в памяти для быстрого доступа и выгружает их на диск по мере роста данных.

● **SSTable** (отсортированные строковые таблицы): Поддерживают данные в виде отсортированных неизменяемых файлов, что позволяет быстро читать данные последовательно и эффективно объединять их.

● **Инвертированный индекс:** Соотносит термины с их местоположениями в документах, что ускоряет полнотекстовый поиск и поиск по ключевым словам.

● **Фильтры Блума:** Обеспечивают вероятностную проверку принадлежности, позволяя быстро исключать неподходящие данные без точного поиска.

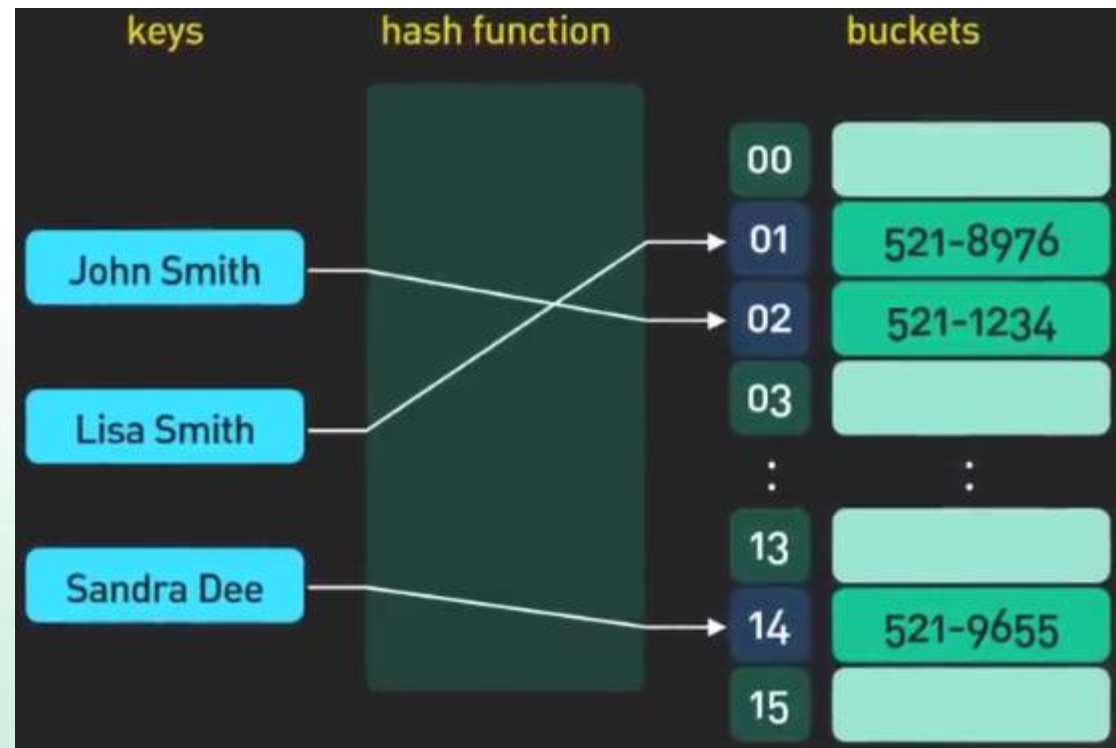
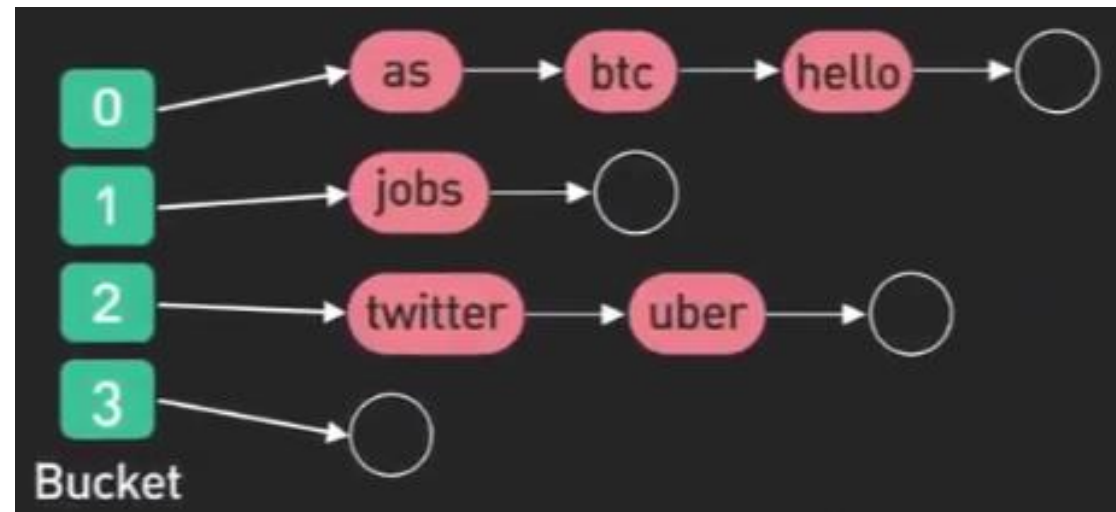
● **Битовые индексы:** Представляют присутствие или отсутствие данных в виде битов, значительно ускоряя логические и аналитические запросы.

● **R-деревья:** Используют пространственно-ориентированные древовидные структуры для эффективного поиска многомерных данных, таких как географические координаты.

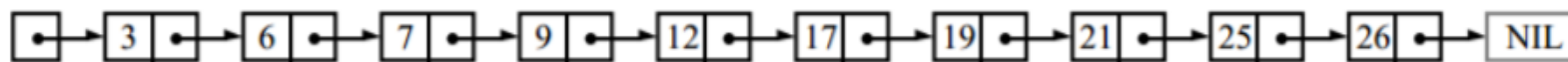
● **Журнал записи** (Write-Ahead Log, WAL): Логирует все изменения перед их применением к основной базе данных, обеспечивая устойчивость к сбоям и быстрое восстановление.

Hash index

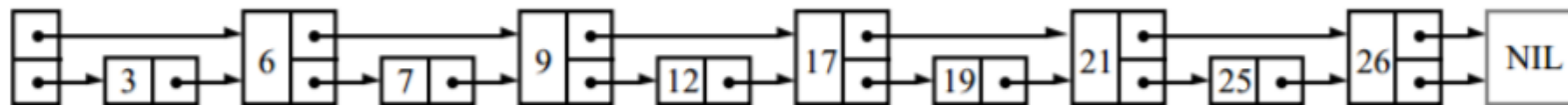
Хеш-индекс — структура данных, реализующая интерфейс ассоциативного массива, а именно, она позволяет хранить пары (ключ, значение) и выполнять три операции: операцию добавления новой пары, операцию удаления и операцию поиска пары по ключу.



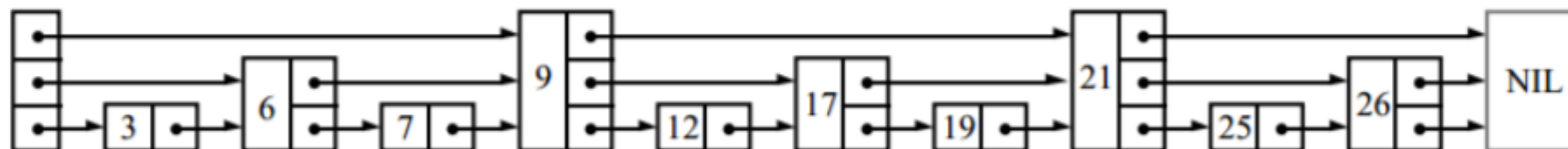
Для поиска элемента в связном списке мы должны просмотреть каждый его узел:



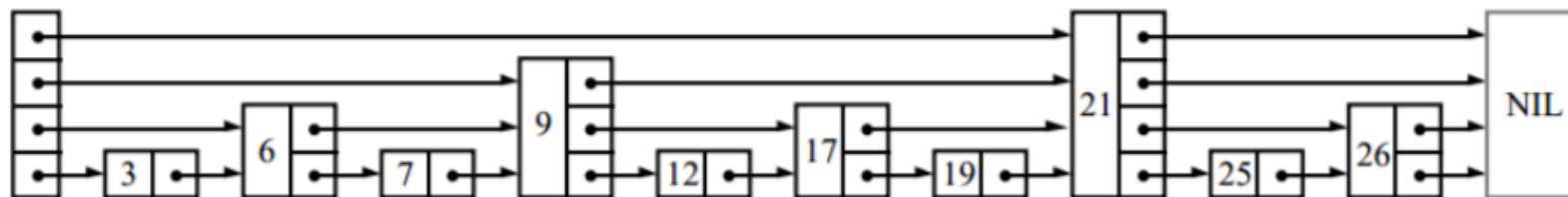
Если список хранится отсортированным и каждый второй его узел дополнительно содержит указатель на два узла вперед, нам нужно просмотреть не более, чем $\lceil n/2 \rceil + 1$ узлов (где n — длина списка):



Аналогично, если теперь каждый четвёртый узел содержит указатель на четыре узла вперёд, то потребуется просмотреть не более чем $\lceil n/4 \rceil + 2$ узла:



Если каждый 2^i -ый узел содержит указатель на 2^i узлов вперёд, то количество узлов, которые необходимо просмотреть, сократится до $\lceil \log_2 n \rceil$, а общее количество указателей в структуре лишь удвоится:



Такая структура данных может использоваться для быстрого поиска, но вставка и удаление узлов будут медленными.

Skip-list

вероятностная структура данных, основанная на нескольких параллельных отсортированных связных списках с эффективностью, сравнимой с двоичным деревом (порядка $O(\log n)$ среднее время для большинства операций).

В основе списка с пропусками лежит расширение отсортированного связного списка дополнительными связями, добавленными в случайных путях с геометрическим/негативным бинаomialным распределением, таким образом, чтобы поиск по списку мог быстро пропускать части этого списка. Вставка, поиск и удаление выполняются за логарифмическое случайное время.

R tree



R-дерево (англ. *R-trees*) — древовидная структура данных (дерево), предложенная в 1984 году Антонином Гуттманом.

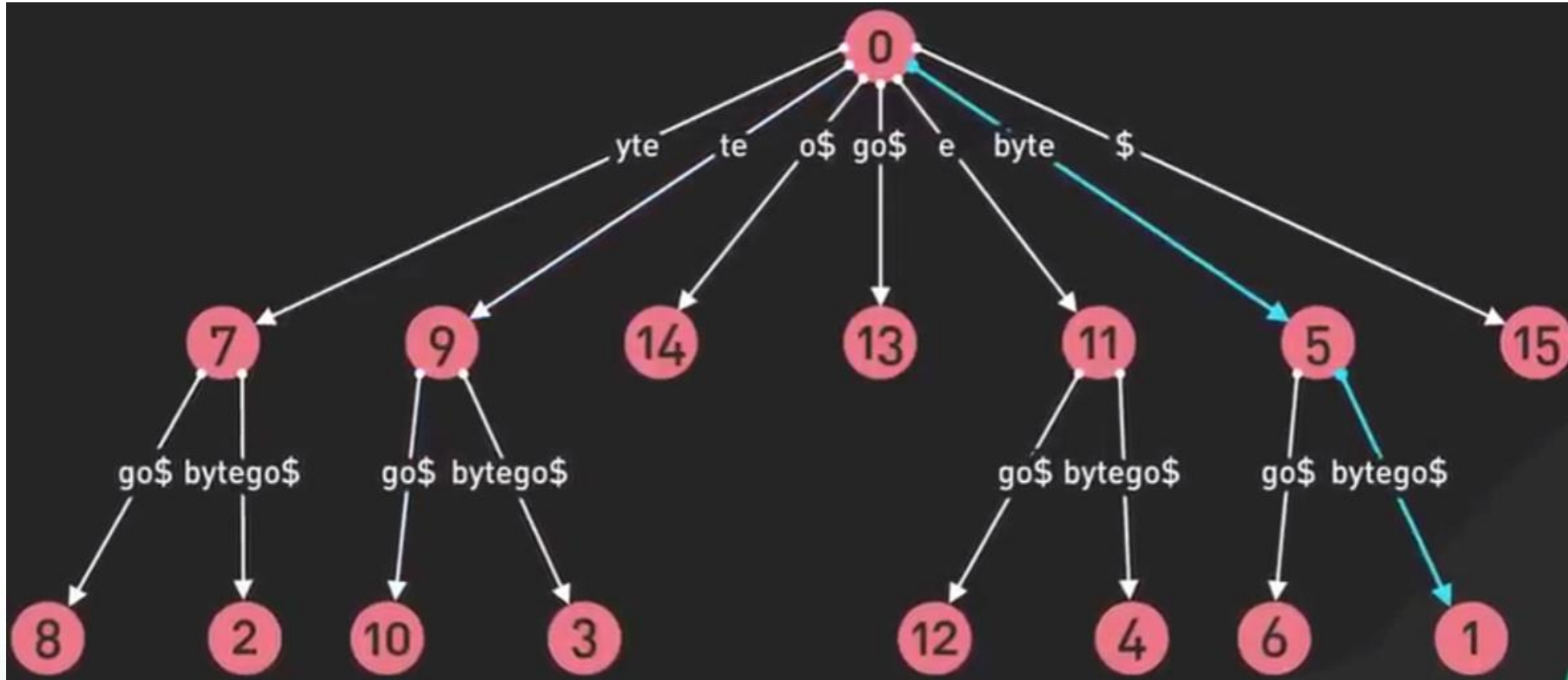
Она подобна B-дереву, но используется для организации доступа к пространственным данным, то есть для индексации многомерной информации, такой, например, как географические данные с двумерными координатами (широтой и долготой).

Типичным запросом с использованием R-деревьев мог бы быть такой:

«Найти все музеи в пределах 2 километров от моего текущего местоположения».

Эта структура данных разбивает многомерное пространство на множество иерархически вложенных и, возможно, пересекающихся, прямоугольников (для двумерного пространства). В случае трехмерного или многомерного пространства это будут прямоугольные параллелепипеды (кубоиды) или параллелотопы.

Suffix tree



В информатике суффиксное **дерево** (также называемое **деревом PAT** или, в более ранней форме, **позиционным деревом**) — это сжатое дерево, содержащее все суффиксы заданного текста в качестве их ключей и позиции в тексте в качестве их значений. Суффиксные деревья позволяют особенно быстро реализовывать многие важные строковые операции.



B-Tree

Это сбалансированное дерево поиска, в котором каждый узел содержит множество ключей и имеет более двух потомков.

Здесь количество ключей в узле и количество его потомков зависит от порядка В-дерева. Каждое В-дерево имеет порядок.

В-дерево порядка m обладает следующими свойствами:

Свойство 1: Глубина всех листьев одинакова.

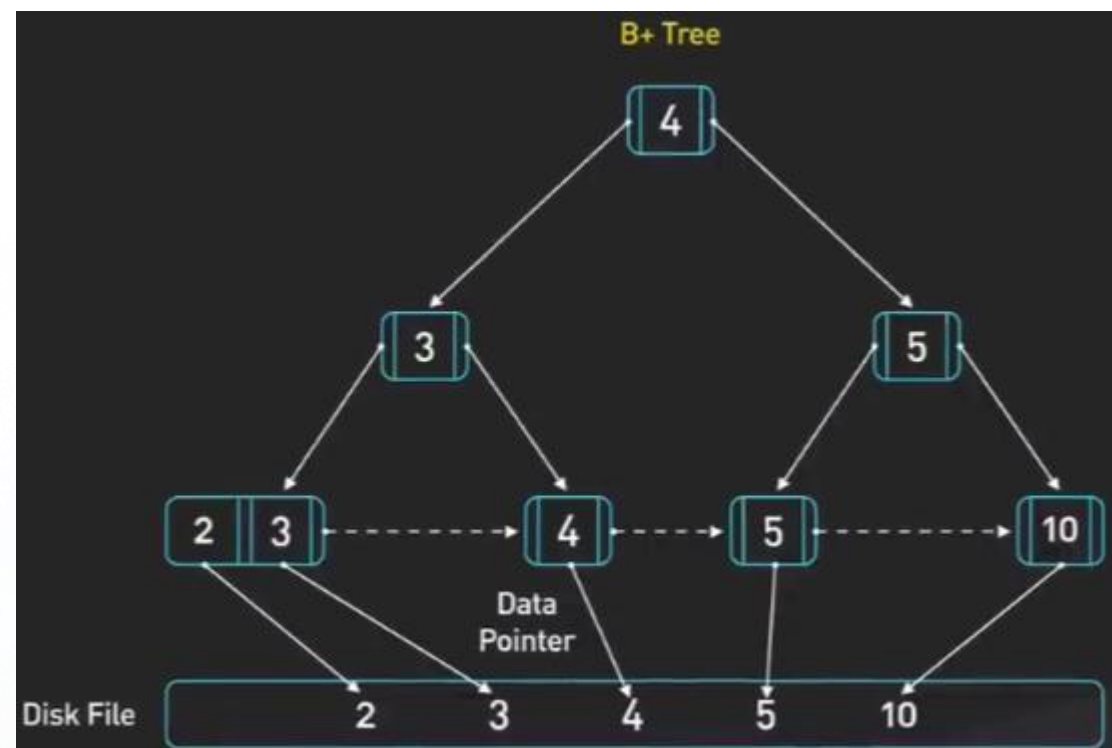
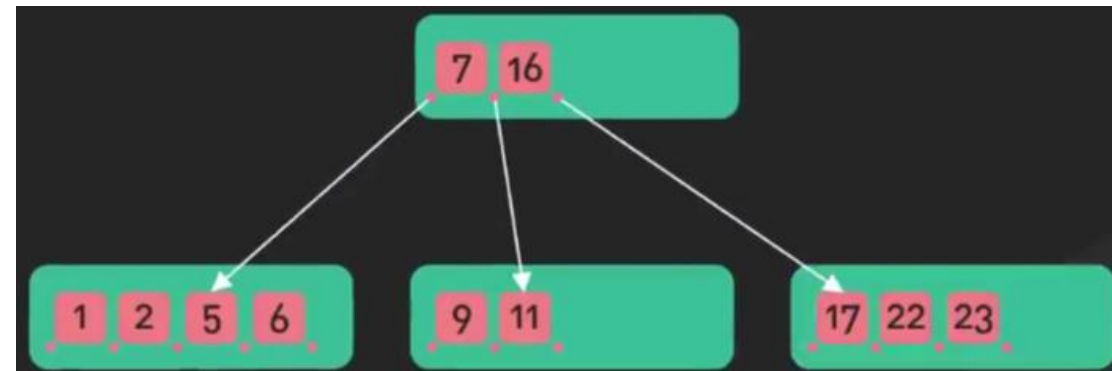
Свойство 2: Все узлы, кроме корня должны иметь как минимум $(m/2) - 1$ ключей и максимум $m-1$ ключей.

Свойство 3: Все узлы без листьев, кроме корня (т.е. все внутренние узлы), должны иметь минимум $m/2$ потомков.

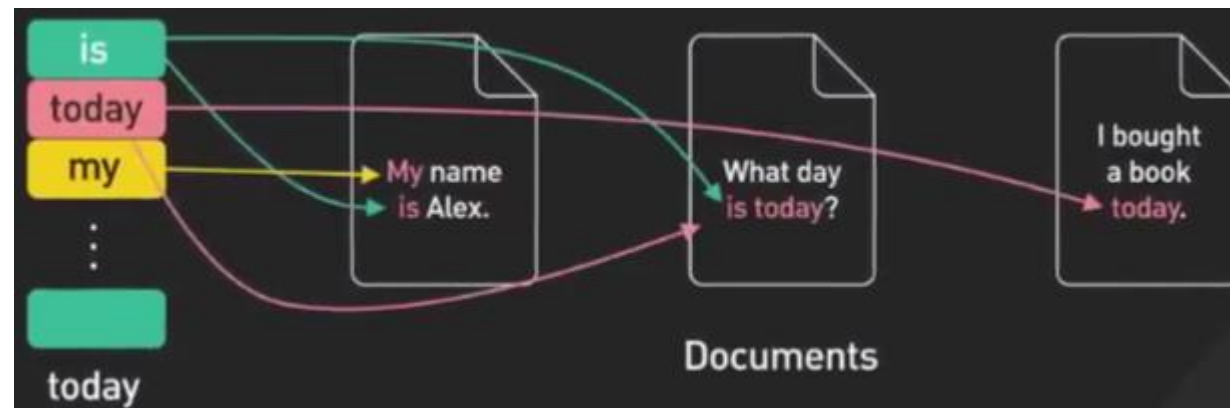
Свойство 4: Если корень – это узел не содержащий листьев, он должен иметь минимум 2 потомка.

Свойство 5: Узел без листьев с $n-1$ ключами должен иметь n потомков.

Свойство 6: Все ключи в узле должны располагаться в порядке возрастания их значений.



Inverted index



Структура данных, в которой для каждого слова коллекции документов в соответствующем списке перечислены все документы в коллекции, в которых оно встретилось.

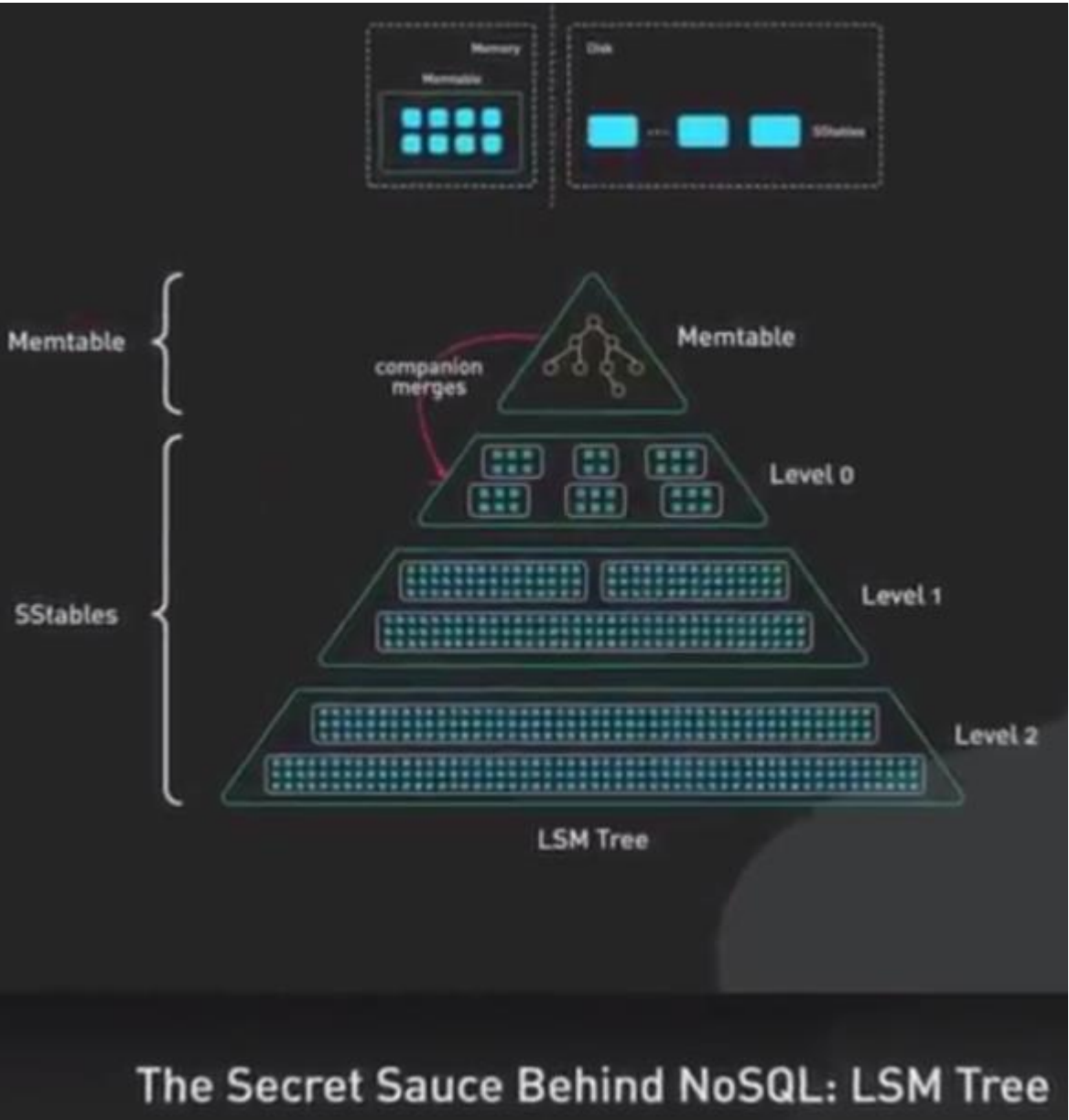
Инвертированный индекс используется для поиска по текстам.

Есть два варианта инвертированного индекса:

- индекс, содержащий только список документов для каждого слова,
- индекс, дополнительно включающий позицию слова в каждом документе.

Структуры данных, которые делают базы данных быстрыми и масштабируемыми:

SSTable и LSM Tree



Спасибо за внимание!